

MI AI API

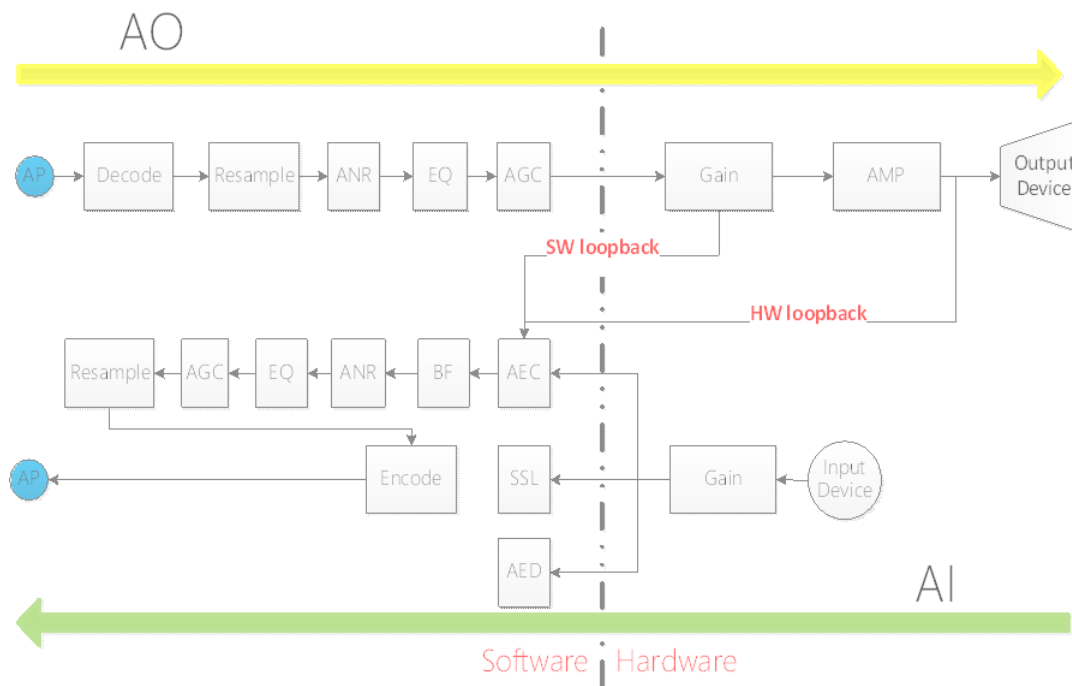
1 Overview

1.1. Module Description

Audio Input (Audio Input, AI) mainly implements functions such as configuring and enabling audio input devices, acquiring audio frame data, and processing acoustic algorithms.

Acoustic algorithm processing mainly includes: resampling, echo cancellation, noise reduction, high-pass filtering, equalizer, automatic gain control, sound event detection, sound source localization, beamforming, etc. In API versions 2.19 and later, MI_AI no longer contains related algorithm functions. Please check the version information in `mi_ai.h` of the SDK.

1.2. Flow Diagram



Picture 1-1

In API versions 2.19 and later, MI_AI/MI_AO no longer contains related algorithm functions, and only maintains basic capture/playback functions.

1.3. Keyword Description

- Device

Different from the Device concept of other modules, AI's Device refers to the path from external devices to the chip audio codec, such as Amic/Dmic/I2S RX/Line in. Because different devices may use the same hardware, whether each device can be used together depends on the specific chip.

- Channel

AI's Channel refers to the number of channels in software concept, and one channel corresponds to several hardware channels, which is determined by MI_AUDIO_SoundMode_e.

- SRC

SRC (Sample Rate Conversion), namely resample, resampling.

- AGC

AGC (Automatic Gain Control), automatic gain control, is used to control the output gain.

- EQ

EQ (Equalizer), equalizer processing, is used to process specific frequency bands.

- ANR

ANR (Acoustic Noise Reduction), noise reduction, is used to remove noise that persists in the environment and has a fixed frequency.

- BF

BF (beamforming), a beamforming algorithm, is used for microphone arrays to enhance sound.

- AEC

AEC (Acoustic Echo Cancellation), echo cancellation.

- SSL

SSL (Sound Source Localization), sound source localization, used for microphone arrays to identify the direction of sound

- AED

AED (Acoustic Event Detection), sound event monitoring, can only detect baby crying and excessive decibel sounds at present.

- HPF

HPF (High-Pass Filtering), high-pass filtering

1.4. Device ID

Table 1-1 AI Dev ID and physical device comparison table of different chips

	0	1	2	3	4
Pretzel	Amic	Dmic	I2s Rx	Line in	
Macaron	Amic	Dmic	I2s Rx	Line in	
Taiyaki	Amic	Dmic	I2s Rx	Line in	
Takoyaki	Amic	Dmic	I2s Rx	Line in	Amic + I2sRx
Pudding	Amic	Dmic	I2s Rx	Line in	I2s Rx + St
Ispahan	Amic	Dmic	I2s Rx	Line in	I2s Rx + St
Ikayaki	Amic(ADC0/1)	Dmic	I2s Rx	Line in(ADC0/1)	Amic(ADC

Table 1-2 Specifications of different series of chips

	Amic/Line in	Dmic	I2s Rx
Pretzel	Support 2-channel 8/16/32/48kHz sampling rate	Support 4-channel 8/16/32/48kHz sampling rate	Support standard I2S mode and TDM mode, TDM mode can be extended to 8 channels, support 4-wire and 6-wire mode, can provide Mclk. Support 8/16/32/48kHz sampling rate.
Macaron	Support 2-channel 8/16/32/48kHz sampling rate	Support 2-channel 8/16/32kHz sampling rate	Only supports standard I2S mode, and can only be used as master, only supports 4-wire mode, and cannot provide Mclk. Support 8/16/32/48kHz sampling rate.
Taiyaki	Support 1-channel 8/16/32/48kHz sampling rate	Support 4-channel 8/16/32/48kHz sampling rate	Only supports standard I2S mode, and can only be used as master, only supports 4-wire mode, and cannot provide Mclk. Support 8/16/32/48kHz sampling rate.
	Amic/Line in	Dmic	I2s Rx

Takoyaki	Support 1 channel 8/16/32/48kHz sampling rate	Support 4-channel 8/16/32/48kHz sampling rate	Only supports standard I2S mode, and can only be used as master, only supports 4-wire mode, and cannot provide Mclk. Support 8/16/32/48kHz sampling rate.
Pudding	Support 2-channel 8/16/32/48kHz sampling rate	Support 2-channel 8/16/32kHz sampling rate	Support standard I2S mode and TDM mode, TDM mode can be extended to 8 channels, support 4-wire and 6-wire mode, can provide Mclk. Support 8/16/32/48kHz sampling rate.
Ispahan	Support 2-channel 8/16/32/48kHz sampling rate	Support 2-channel 8/16/32kHz sampling rate	Only supports standard I2S mode, and can only be used as master, only supports 4-wire mode, and cannot provide Mclk. Support 8/16/32/48kHz sampling rate.
Ikayaki	Support 3-channel 8/16/32/48kHz sampling rate	Support 4-channel 8/16/32/48kHz sampling rate	Support standard I2S mode and TDM mode, TDM mode can be extended to 8 channels, support 4-wire and 6-wire mode, can provide Mclk. Support 8/16/32/48kHz sampling rate.

2. API Reference

2.1. Function Module API

table 2-1

API name	Features
MI_AI_SetPubAttr [#22-mi_ai_setpubattr]	Set AI device properties
API name MI_AI_GetPubAttr [#23-mi_ai_getpubattr]	Features Get AI device properties

MI_AI_Enable [#24-mi_ai_enable]	Enable AI devices
MI_AI_Disable [#25-mi_ai_disable]	Disable AI devices
MI_AI_EnableChn [#26-mi_ai_enablechn]	Enable AI channel
MI_AI_DisableChn [#27-mi_ai_disablechn]	Disable AI channel
MI_AI_GetFrame [#28-mi_ai_getframe]	get audio frame
MI_AI_ReleaseFrame [#29-mi_ai_releaseframe]	release audio frame
MI_AI_SetChnParam [#210-mi_ai_setchnparam]	设置AI通道参数
MI_AI_GetChnParam [#211-mi_ai_getchnparam]	获取AI通道参数
MI_AI_EnableReSmp [#212-mi_ai_enableresmp]	启用AI 重采样
MI_AI_DisableReSmp [#213-mi_ai_disableresmp]	禁用AI 重采样
MI_AI_SetVqeAttr [#214-mi_ai_setvqeattr]	设置AI通道的声音质量增强功能相关属性
MI_AI_GetVqeAttr [#215-mi_ai_getvqeattr]	获取AI通道的声音质量增强功能相关属性
MI_AI_EnableVqe [#216-mi_ai_enablevqe]	使能AI通道的声音质量增强功能
MI_AI_DisableVqe [#217-mi_ai_disablevqe]	禁用AI通道的声音质量增强功能
MI_AI_ClrPubAttr [#218-mi_ai_clrpubattr]	清除AI设备属性
MI_AI_SaveFile [#219-mi_ai_savefile]	开启音频输入保存文件功能
MI_AI_SetVqeVolume [#220-mi_ai_setvqevolume]	设置AI通道的音量大小
MI_AI_GetVqeVolume [#221-mi_ai_getvqevolume]	获取AI通道的音量大小
MI_AI_SetAencAttr [#222-mi_ai_setaencattr]	设置AI通道编码功能相关属性
API name	Features
MI_AI_GetAencAttr [#223-mi_ai_getaencattr]	获取AI通道编码功能相关属性

MI_AI_GetExtAecChn [#231-mi_ai_setextaecchn]	设置AI通道回声消除功能参考的外部AI通道
MI_AI_SetSslInitAttr [#232-mi_ai_setsslinitattr]	设置AI通道声源定位功能的初始化参数
MI_AI_GetSslInitAttr [#233-mi_ai_getsslinitattr]	获取AI通道声源定位功能的初始化参数
MI_AI_SetSslConfigAttr [#234-mi_ai_setsslconfigattr]	设置AI通道声源定位功能的配置参数
MI_AI_GetSslConfigAttr [#235-mi_ai_getsslconfigattr]	获取AI通道声源定位功能的配置参数
MI_AI_EnableSsl [#236-mi_ai_enablesll]	使能AI通道声源定位功能
MI_AI_DisableSsl [#237-mi_ai_disablesll]	禁止AI通道声源定位功能
MI_AI_GetSslDoa [#238-mi_ai_getssldoa]	获取AI通道声源定位功能的结果
MI_AI_SetBfInitAttr [#239-mi_ai_setbfinitattr]	设置AI通道波束成形成功能的初始化参数
MI_AI_GetBfInitAttr [#240-mi_ai_getbfinitattr]	获取AI通道波束成形成功能的初始化参数
MI_AI_SetBfConfigAttr [#241-mi_ai_setbfconfigattr]	设置AI通道波束成形成功能的配置参数
API name	Features
MI_AI_GetBfConfigAttr [#242-mi_ai_getbfconfigattr]	获取AI通道波束成形成功能的配置参数

MI_AI_GetBfConfigAttr [#242-mi_ai_getbfconfigattr]	获取AI通道波束成形功能的配置参数
MI_AI_EnableBf [#243-mi_ai_enablebf]	使能AI通道波束成形功能
MI_AI_DisableBf [#244-mi_ai_disablebf]	禁止AI通道波束成形功能
MI_AI_SetBfAngle [#245-mi_ai_setbfangle]	设置AI通道波束成形功能的作用角度
MI_AI_SetMute [#246-mi_ai_setmute]	设置AI通道的静音参数
MI_AI_GetMute [#247-mi_ai_getmute]	获取AI通道的静音参数
MI_AI_InitDev [#248-mi_ai_initdev]	初始化AI设备
MI_AI_DeInitDev [#249-mi_ai_deinitdev]	反初始化AI设备
MI_AI_DupChn [#250-mi_ai_dupchn]	同步AI通道状态
MI_AI_DevIsEnable [#251-mi_ai_devisenable]	获取AI设备的使能状态
MI_AI_ChnlIsEnable [#252-mi_ai_chnisenable]	获取AI通道的使能状态

2.2. MI_AI_SetPubAttr

- 功能
设置AI设备属性。
- 语法

```
MI_S32 MI_AI_SetPubAttr(MI_AUDIO_DEV AiDevId,MI_AUDIO_Attr_t *pstAttr);
```

- 形参
表2-2

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
pstAttr	AI设备属性指针	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.a/libmi_ai.so

- 注意

音频输入设备的属性决定了输入数据的格式，输入设备属性包括采样率、采样精度、工作模式、通道的数据格式、每帧的采样点数、通道数目和I2S的配置参数。若需要对接Codec，这些属性应与待对接Codec 要求一致。

- 采样率

采样率指一秒内的采样点数，采样率越高表明失真度越小，处理的数据量也就随之增加。一般来说语音使用8k 采样率，音频使用32k或以上的采样率；目前仅支持8/16/32/48kHz采样率。若需要对接Codec，设置时请确认对接的Audio Codec 是否支持所要设定的采样率。

- 采样精度

采样精度指某个通道的采样点数据宽度。目前采样精度仅支持16bit。

- 工作模式

音频输入目前支持I2S主模式、I2S从模式、Tdm主模式、Tdm从模式，但芯片支持的工作模式不尽相同。

- 通道的数据格式

数据格式表示通道中的数据排列方式。**Mono**表示AI通道的数据是一个物理通道数据。**Stereo**表示AI通道的数据是两个物理通道数据交织在一起，组成立体声。**Queue**表示一个通道中有多个物理通道数据首尾相接存放。在**Queue**模式下启用算法时，所有通道使用相同的算法参数。

- 每帧的采样点数

当音频采样率较高时，建议相应地增加每帧的采样点数目。若发现采集到的声音断续，则需要增大每帧的采样点数和通道配置的buffer数目。

- 通道数目

通道数目是指当前输入设备在软件概念中的通道数。一个软件上的通道有多少个物理通道由通道的数据格式所决定。比如通道的数据格式为Mono/Stereo，则一个软件通道对 $\frac{1}{2}$ 个物理通道。

- I2S的配置参数

I2S的配置参数指定I2S Mclk的频率、I2S传输的数据格式、I2S使用的是4-wire mode还是6-wire mode、I2S的Tdm slot数以及I2S收发数据的位宽。

- 举例

下面的代码实现从AI通道取一帧数据，然后释放的功能。

```
MI_S32 ret;

MI_AUDIOIO_Attr_t stAttr;

MI_AUDIOIO_Dev AiDevId = 0;

MI_AI_CHN AiChn = 0;

MI_SYS_ChnPort_t stChnPort;

MI_S32 s32Fd;

fd_set readFdSet;

struct timeval stTimeOut;

MI_AUDIOIO_Frame_t stAiChFrame;

MI_AUDIOIO_AecFrame_t stAecFrame;

MI_SYS_ChnPort_t stAiChnOutputPort;

MI_SYS_Init();

stAttr.eBitwidth = E_MI_AUDIOIO_BIT_WIDTH_16;

stAttr.eSamplerate = E_MI_AUDIOIO_SAMPLE_RATE_8000;
```

```
stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;

stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;

stAttr.u32PtNumPerFrm = 160;

stAttr.u32ChnCnt = 1;

/* set public attribute of AI device */

ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

if (MI_SUCCESS != ret)
{
    printf("set ai %d attr err:0x%x\n", AiDevId, ret);

    return ret;
}

/* get public attribute of AI device */

ret = MI_AI_GetPubAttr(AiDevId, &stAttr);

if (MI_SUCCESS != ret)
{
    printf("get ai %d attr err:0x%x\n", AiDevId, ret);

    return ret;
}

/* enable AI device */

ret = MI_AI_Enable(AiDevId);

if (MI_SUCCESS != ret)
{
    printf("enable ai %d err:0x%x\n", AiDevId, ret);

    return ret;
}
```

```

/* enable AI Channel */

ret = MI_AI_EnableChn(AiDevId, AiChn);

if (MI_SUCCESS != ret)
{

printf("enable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

    return ret;
}

/* set buffer depth */

stAiChnOutputPort.eModId = E_MI_MODULE_ID_AI;

stAiChnOutputPort.u32DevId = AiDevId;

stAiChnOutputPort.u32ChnId = AiChn;

stAiChnOutputPortu32PortId = 0;

MI_SYS_SetChnOutputPortDepth(&stAiChnOutputPort, 1, 8);

/* get port fd */

stChnPort.eModId = E_MI_MODULE_ID_AI;

stChnPort.u32DevId = AiDevId;

stChnPort.u32ChnId = AiChn;

stChnPort.u32PortId = 0;

ret = MI_SYS_GetFd(&stChnPort, &s32Fd);

if (MI_SUCCESS != ret)
{

printf("Dev%d Chn%d failed to call MI_SYS_GetFd!!!\n", AiDevId, A

    return ret;

```



```

}

/* select 100ms */
FD_ZERO(&readFdSet);
FD_SET(s32Fd, &readFdSet);

stTimeOut.tv_sec = 0;
stTimeOut.tv_usec = 100 * 1000;

ret = select(s32Fd + 1, &readFdSet, NULL, NULL, &stTimeOut);
if (FD_ISSET(s32Fd, &readFdSet))
{

ret = MI_AI_GetFrame(AiDevId, AiChn, &stAiChFrame, &stAecFrame, 0

    if (MI_SUCCESS == ret)
    {

        /* do something */

MI_AI_ReleaseFrame(AiDevId, AiChn, &stAiChFrame, &stAecFrame);

    }
}

/* disable AI Channel */
ret = MI_AI_DisableChn(AiDevId, AiChn);

if (MI_SUCCESS != ret)
{

printf("disable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

    return ret;
}

```

```

}

/* disable AI Device */

ret = MI_AI_Disable(AiDevId);

if (MI_SUCCESS != ret)
{
    printf("disable ai %d err:0x%x\n", AiDevId, ret);

    return ret;
}

MI_SYS_Exit();

```

2.3. MI_AI_GetPubAttr

- 功能

获取AI设备属性。

- 语法

```
MI_S32 MI_AI_GetPubAttr(MI_AUDIO_DEV AiDevId, MI_AUDIO_Attr_t *pstAttr);
```

- 形参

表2-3

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
pstAttr	AI设备属性指针	输出

- 返回值

- 0 成功。

- 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: `mi_ai.h`
 - 库文件: `libmi_ai.a/libmi_ai.so`
- 注意
 - 获取的属性为当前配置的属性。
 - 如果从来没有配置过属性，则返回失败。
- 举例

请参考[MI_AI_SetPubAttr](#) [#22-mi_ai_setpubattr]举例部分。

2.4. MI_AI_Enable

- 功能
启用AI设备。
- 语法

```
MI_S32 MI_AI_Enable(MI_AUDIO_DEV AiDevId);
```

- 形参

表2-4

参数名称	描述	输入/输出
AiDevId	音频设备号	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: `mi_ai.h`

- 库文件: libmi_ai.a/libmi_ai.so
- 注意
 - 必须在启用前配置AI设备属性，否则返回属性未配置错误。
 - 如果AI设备已经处于启用状态，则直接返回成功。
- 举例

请参考[MI_AI_SetPubAttr](#) [#22-mi_ai_setpubattr]举例部分。

2.5. MI_AI_Disable

- 功能
禁用AI设备。
- 语法

```
MI_S32 MI_AI_Disable(MI_AUDIO_DEV AiDevId);
```

- 形参

表2-5

参数名称	描述	输入/输出
AiDevId	音频设备号	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so
- 注意
 - 如果AI设备已经处于禁用状态，则直接返回成功。

- 禁用AI设备前必须先禁用该设备下已启用的所有AI通道。
- 举例

请参考[MI_AI_SetPubAttr](#) [#22-mi_ai_setpubattr]举例部分。

2.6. MI_AI_EnableChn

- 功能
启用AI通道
- 语法

```
MI_S32 MI_AI_EnableChn(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-6

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so

- 注意

启用AI通道前，必须先启用其所属的AI设备，否则返回设备未启动的错误码

- 举例

请参考[MI_AI_SetPubAttr](#) [#22-mi_ai_setpubattr]举例部分。

2.7. MI_AI_DisableChn

- 功能
禁用AI通道
- 语法

```
MI_S32 MI_AI_DisableChn( AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-7

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: `mi_ai.h`
 - 库文件: `libmi_ai.a/libmi_ai.so`
- 注意
 - 如果AI通道已经处于禁用状态，则直接返回成功。
 - 禁用AI通道前必须先禁用该AI通道已经使能的音频算法。
- 举例

请参考[MI_AI_SetPubAttr](#) [#22-mi_ai_setpubattr]举例部分。

2.8. MI_AI_GetFrame

- 功能
 获取音频帧
- 语法

```
MI_S32 MI_AI_GetFrame(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn,
MI_AUDIO_Frame_t *pstFrm, MI_AUDIO_AecFrame_t *pstAecFrm ,
MI_S32 s32MilliSec);
```

- 形参

表2-8

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定	输入
pstFrm	音频帧结构体指针	输出
pstAecFrm	回声消除参考帧结构体指针	输出
s32MilliSec	获取数据的超时时间 -1 表示阻塞模式，无数据时一直等待； 0 表示非阻塞模式，无数据时则报错返回； >0表示阻塞s32MilliSec毫秒，超时则报错返回	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h

- 库文件: libmi_ai.a/libmi_ai.so
- 注意
 - 在获取音频帧前，必须先使能AI通道，否则返回错误。
 - 如果需要获取回声消除参考帧，pstAecFrm 不能是空指针，如果不想获取回声抵消参考帧pstAecFrm置为空指针即可。
 - s32MilliSec 的值必须大于等于-1，等于-1时采用阻塞模式获取数据，等于0时采用非阻塞模式获取数据，大于0时，阻塞s32MilliSec 毫秒后，没有数据则返回超时并报错。
 - 本接口支持select/poll 操作，建议使用select/poll操作代替s32MilliSec。
- 举例

请参考[MI_AI_SetPubAttr](#) [#22-mi_ai_setpubattr]举例部分。

2.9. MI_AI_ReleaseFrame

- 功能
释放音频帧
- 语法

```
MI_S32 MI_AI_ReleaseFrame(MI_AUDIO_DEV AiDevId,    MI_AI_CHN
AiChn,MI_AUDIO_Frame_t *pstFrm, MI_AUDIO_AecFrame_t
*pstAecFrm);
```

- 形参

表2-9

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstFrm	音频帧结构体指针	输入
pstAecFrm	回声抵消参考帧结构体指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so
- 举例

请参考[MI_AI_SetPubAttr](#) [#22-mi_ai_setpubattr]举例部分。

2.10. MI_AI_SetChnParam

- 功能
设置AI通道参数
- 语法

```
MI_S32 MI_AI_SetChnParam( AiDevId, MI_AI_CHN AiChn,  
MI_AI_ChnParam_t *pstChnParam);
```

- 形参

表2-10

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定	输入
pstChnParam	音频通道参数结构体指针	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [jump]。

- 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.a/libmi_ai.so

- 注意

需要设置AI通道参数，必须先配置和使能AI 设备。

- 举例

下面的代码实现配置和获取AI通道参数。

```
MI_S32 ret;

MI_AUDIO_DEV AiDevId = 0;

MI_AI_CHN AiChn = 0;

MI_AUDIO_Attr_t stAttr;

MI_AI_ChnParam_t stChnParam;

MI_SYS_Init();

memset(&stAttr, 0x0, sizeof(MI_AUDIO_Attr_t));

stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;

stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;
```

```
stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;

stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;

stAttr.u32PtNumPerFrm = 160;

stAttr.u32ChnCnt = 1;

/* set public attribute of AI device */

ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

if (MI_SUCCESS != ret)

{

    printf("set Dev%d attr err:0x%x\n", AiDevId, ret);

    return ret;

}

/* enable AI device */

ret = MI_AI_Enable(AiDevId);

if (MI_SUCCESS != ret)

{

    printf("enable Dev%d err:0x%x\n", AiDevId, ret);

    return ret;

}

/* enable AI Channel */

ret = MI_AI_EnableChn(AiDevId, AiChn);

if (MI_SUCCESS != ret)

{

    printf("enable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

    return ret;

}
```

```

}

memset(&stChnParam, 0x0, sizeof(stChnParam));

stChnParam.stChnGain.bEnableGainSet = TRUE;

stChnParam.stChnGain.s16FrontGain = 0;

stChnParam.stChnGain.s16RearGain = 0;

ret = MI_AI_SetChnParam(AiDevId, AiChn, &stChnParam);

if (MI_SUCCESS != ret)
{

printf("set Dev%d Chn%d param err:0x%x\n", AiDevId, AiChn, ret);

    return ret;

}

/* get channel param */

ret = MI_AI_GetChnParam(AiDevId, AiChn, &stChnParam);

if (MI_SUCCESS != ret)
{

printf("get Dev%d Chn%d param err:0x%x\n", AiDevId, AiChn, ret);

    return ret;

}

/* disable AI Channel */

ret = MI_AI_DisableChn(AiDevId, AiChn);

if (MI_SUCCESS != ret)
{

```

```

printf("disable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

    return ret;

}

/* disable AI Device */

ret = MI_AI_Disable(AiDevId);

if (MI_SUCCESS != ret)

{

    printf("disable ai %d err:0x%x\n", AiDevId, ret);
    return ret;

}

MI_SYS_Exit();

```

2.11. MI_AI_GetChnParam

- 功能

获取AI通道参数

- 语法

```

MI_S32 MI_AI_GetChnParam(MI_AUDIO_DEV AiDevId,    MI_AI_CHN
AiChn, MI_AI_ChnParam_t *pstChnParam);

```

- 形参

表2-11

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstChnParam	音频通道参数结构体指针。	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so

- 举例

请参考[MI_AI_SetChnParam](#) [#210-mi_ai_setchnparam]举例部分。

2.12. MI_AI_EnableReSmp

- 功能

启用AI 重采样
- 语法

```
MI_S32 MI_AI_EnableReSmp(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_AUDIO_SampleRate_e eOutSampleRate);
```

- 形参

表2-12

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
eOutSampleRate	音频重采样的输出采样率。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so libSRC_LINUX.so
- 注意
 - 在启用AI通道之后，调用此接口启用重采样功能。
 - 若需要使能波束成形功能，需在使能波束成形功能后再启用重采样。
 - 在API版本2.19及以后，已不支持此接口。
- 举例

下面的代码实现AI 从8K 到16K 的重采样

```
MI_S32 ret;

MI_AUDIO_DEV AiDevId = 0;

MI_AI_CHN AiChn = 0;

MI_AUDIO_Attr_t stAttr;

MI_SYS_Init();

memset(&stAttr, 0x0, sizeof(MI_AUDIO_Attr_t));

stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
```

```
stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;

stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;

stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;

stAttr.u32PtNumPerFrm = 160;

stAttr.u32ChnCnt = 1;

/* set public attribute of AI device */

ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

if (MI_SUCCESS != ret)

{

    printf("set Dev%d attr err:0x%x\n", AiDevId, ret);

    return ret;

}

/* enable AI device */

ret = MI_AI_Enable(AiDevId);

if (MI_SUCCESS != ret)

{

    printf("enable Dev%d err:0x%x\n", AiDevId, ret);

    return ret;

}

/* enable AI Channel */

ret = MI_AI_EnableChn(AiDevId, AiChn);

if (MI_SUCCESS != ret)

{

    printf("enable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);
```



```

        return ret;

    }

    ret = MI_AI_EnableReSmp(AiDevId, AiChn, E_MI_AUDIO_SAMPLE_RATE_16);

    if (MI_SUCCESS != ret)
    {

        printf("resample Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

        return ret;

    }

    ret = MI_AI_DisableReSmp(AiDevId, AiChn);

    if (MI_SUCCESS != ret)
    {

        printf("disable resample Dev%d Chn%d err:0x%x\n", AiDevId, AiChn,

            return ret;

        }

        /* disable AI Channel */

        ret = MI_AI_DisableChn(AiDevId, AiChn);

        if (MI_SUCCESS != ret)
        {

            printf("disable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

            return ret;

        }

        /* disable AI Device */

```

```
ret = MI_AI_Disable(AiDevId);

if (MI_SUCCESS != ret)
{
    printf("disable ai %d err:0x%x\n", AiDevId, ret);

    return ret;
}

MI_SYS_Exit();
```

2.13. MI_AI_DisableReSmp

- 功能

禁用AI 重采样

- 语法

```
MI_S32 MI_AI_DisableReSmp( AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-13

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so libSRC_LINUX.so`
- 注意
 - 不再使用AI 重采样功能的话，应该调用此接口将其禁用。
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_EnableReSmp](#) [#212-mi_ai_enableresmp]举例部分

2.14. MI_AI_SetVqeAttr

- 功能
设置AI的声音质量增强功能相关属性
- 语法

```
MI_S32 MI_AI_SetVqeAttr(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn,  
MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn, MI_AI_VqeConfig_t  
*pstVqeConfig);
```

- 形参

表2-14

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
AoDevId	用于回声抵消的AO设备号	输入
AoChn	用于回声抵消的AO通道号。支持的通道范围由AO设备属性中的最大通道个数u32ChnCnt决定。	输入
pstVqeConfig	音频输入声音质量增强配置结构体指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so libAEC_LINUX.so libAPC_LINUX.so
- 注意
 - 设置AI的声音质量增强功能相关属性前，必须先使能对应的AI通道。
 - Vqe包含的所有算法均支持8/16K采样率，仅ANR/AGC/EQ支持48K。
 - 在API版本2.19及以后，已不支持此接口。
- 举例

下面代码实现使能和禁用声音质量增强属性

```
MI_S32 ret;

MI_AUDIO_DEV AiDevId = 0;

MI_AI_CHN AiChn = 0;

MI_AUDIO_Attr_t stAttr;
```

```

MI_AI_VqeConfig_t stVqeConfig;

MI_U32 u32AecSupfreq[] = {4,6,36,49,50,51};

MI_U32 u32AecSupIntensity[] = {5,4,4,5,10,10,10};

MI_S16 s16Compression_ratio_input[] = {-80, -60, -40, -20, 0};

MI_S16 s16Compression_ratio_output[] = {-80, -60, -30, -15, -10};


MI_SYS_Init();

memset(&stAttr, 0x0, sizeof(MI_AUDIO_Attr_t));

stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;

stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;

stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;

stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;

stAttr.u32PtNumPerFrm = 160;

stAttr.u32ChnCnt = 1;

/* set public attribute of AI device */

ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

if (MI_SUCCESS != ret)

{

    printf("set Dev%d attr err:0x%xn", AiDevId, ret);

    return ret;

}

/* enable AI device */

ret = MI_AI_Enable(AiDevId);

if (MI_SUCCESS != ret)

{

```

```
    printf("enable Dev%d err:0x%x\n", AiDevId, ret);

    return ret;
}

/* enable AI Channel */

ret = MI_AI_EnableChn(AiDevId, AiChn);

if (MI_SUCCESS != ret)
{

printf("enable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

    return ret;
}

/* set vqe attr */

memset(&stVqeConfig, 0x0, sizeof(stVqeConfig));

stVqeConfig.bHpfOpen = TRUE;

stVqeConfig.bAecOpen = TRUE;

stVqeConfig.bAnrOpen = TRUE;

stVqeConfig.bAgcOpen = TRUE;

stVqeConfig.bEqOpen = TRUE;

stVqeConfig.u32ChnNum = 1;

stVqeConfig.s32WorkSampleRate = 8000;

stVqeConfig.s32FrameSample = 128;

stVqeConfig.stHpfCfg.eMode = E_MI_AUDIO_ALGORITHM_MODE_USER;

stVqeConfig.stHpfCfg.eHpfFreq = E_MI_AUDIO_HPF_FREQ_150;

stVqeConfig.stAecCfg.bComfortNoiseEnable = TRUE;

stVqeConfig.stAecCfg.s16DelaySample = 0;
```

```

memcpy(stVqeConfig.stAecCfg.u32AecSupfreq, u32AecSupfreq, **sizeof
(u32AecSupfreq));

memcpy(stVqeConfig.stAecCfg.u32AecSupIntensity, u32AecSupIntensit
(u32AecSupIntensity));

stVqeConfig.stAnrCfg.eMode = E_MI_AUDIO_ALGORITHM_MODE_MUSIC;

stVqeConfig.stAnrCfg.u32NrIntensity = 30;

stVqeConfig.stAnrCfg.u32NrSmoothLevel = 10;

stVqeConfig.stAnrCfg.eNrSpeed = E_MI_AUDIO_NR_SPEED_MID;

stVqeConfig.stAgcCfg.eMode = E_MI_AUDIO_ALGORITHM_MODE_USER;

stVqeConfig.stAgcCfg.stAgcGainInfo.s32GainMax = 15;

stVqeConfig.stAgcCfg.stAgcGainInfo.s32GainMin = 0;

stVqeConfig.stAgcCfg.stAgcGainInfo.s32GainInit = 0;

stVqeConfig.stAgcCfg.u32DropGainMax = 55;

stVqeConfig.stAgcCfg.u32AttackTime = 1;

stVqeConfig.stAgcCfg.u32ReleaseTime = 6;

memcpy(stVqeConfig.stAgcCfg.s16Compression_ratio_input, s16Compre

memcpy(stVqeConfig.stAgcCfg.s16Compression_ratio_output, s16Compr

stVqeConfig.stAgcCfg.s32TargetLevelDb = -3;

stVqeConfig.stAgcCfg.s32NoiseGateDb = -80;

stVqeConfig.stAgcCfg.u32NoiseGateAttenuationDb = 0;

stVqeConfig.stEqCfg.eMode = E_MI_AUDIO_ALGORITHM_MODE_USER;

ret = MI_AI_SetVqeAttr(AiDevId, AiChn, 0, 0, &stVqeConfig);

if (MI_SUCCESS != ret)

{

```

```
printf("set vqe attr Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

    return ret;
}

ret = MI_AI_GetVqeAttr(AiDevId, AiChn, 0, 0, &stVqeConfig);
if (MI_SUCCESS != ret)
{

printf("get vqe attr Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

    return ret;
}

/* enable vqe */
ret = MI_AI_EnableVqe(AiDevId, AiChn);
if (MI_SUCCESS != ret)
{

printf("enable vqe Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

    return ret;
}

/* disable vqe */
ret = MI_AI_DisableVqe(AiDevId, AiChn);
if (MI_SUCCESS != ret)
{

printf("disable vqe Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret)
```



```

        return ret;
    }

    /* disable AI Channel */

    ret = MI_AI_DisableChn(AiDevId, AiChn);

    if (MI_SUCCESS != ret)
    {

        printf("disable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

        return ret;
    }

    /* disable AI Device */

    ret = MI_AI_Disable(AiDevId);

    if (MI_SUCCESS != ret)
    {

        printf("disable ai %d err:0x%x\n", AiDevId, ret);

        return ret;
    }

    MI_SYS_Exit();

```

2.15. MI_AI_GetVqeAttr

- 功能

获取AI的声音质量增强功能相关属性。

- 语法

```

MI_S32 MI_AI_GetVqeAttr(MI_AUDIO_DEV AiDevId,    MI_AI_CHN
AiChn, MI_AI_VqeConfig_t *pstVqeConfig);

```

- 形参

表2-15

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstVqeConfig	音频输入声音质量增强配置结构体指针	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: `mi_ai.h`
 - 库文件: `libmi_ai.a/libmi_ai.so libAEC_LINUX.so libAPC_LINUX.so`
- 注意
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetVqeAttr](#) [#214-mi_ai_setvqeattr]举例部分。

2.16. MI_AI_EnableVqe

- 功能

使能AI的声音质量增强功能。
- 语法

```
MI_S32 MI_AI_EnableVqe(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-16

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [jump]。

- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so libAPC_LINUX.so libAEC_LINUX.so`

- 注意

- 启用声音质量增强功能前必须先使能相对应AI通道和设置相对应AI通道的声音质量增强功能相关属性。
- 多次使能相同AI 通道的声音质量增强功能时，返回成功。
- 在API版本2.19及以后，已不支持此接口。

- 举例

请参考[MI_AI_SetVqeAttr](#) [214-mi_ai_setvqeattr]举例部分。

2.17. MI_AI_DisableVqe

- 功能

禁用AI的声音质量增强功能。

- 语法

```
MI_S32 MI_AI_DisableVqe(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-17

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so libAPC_LINUX.so libAPC_LINUX.so
- 注意
 - 不再使用AI声音质量增强功能时，应该调用此接口将其禁用。
 - 多次禁用相同AI 通道的声音质量增强功能，返回成功。
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetVqeAttr](#) [#214-mi_ai_setvqeattr]举例部分。

2.18. MI_AI_ClrPubAttr

- 功能

清除AI设备属性。
- 语法

```
MI_S32 MI_AI_ClrPubAttr(MI_AUDIO_DEV AiDevId);
```

- 形参

表2-18

参数名称	描述	输入/输出
AiDevId	音频设备号	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so
- 注意
- 举例

下面的代码实现了清除设备属性。

```
MI_S32 ret;

MI_AUDIO_DEV AiDevId = 0;

MI_AI_CHN AiChn = 0;

MI_AUDIO_Attr_t stAttr;

memset(&stAttr, 0x0, sizeof(MI_AUDIO_Attr_t));

stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;

stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;

stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;

stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;
```

```

stAttr.u32PtNumPerFrm = 160;

stAttr.u32ChnCnt = 1;

/* set public attribute of AI device */

ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

if (MI_SUCCESS != ret)
{
    printf("set Dev%d attr err:0x%x\n", AiDevId, ret);

    return ret;
}

/* clean public attr of AI device*/

ret = MI_AI_ClrPubAttr(AiDevId);

if (MI_SUCCESS != ret)
{
    printf("clean Dev%d attr err:0x%x\n", AiDevId, ret);

    return ret;
}

```

2.19. MI_AI_SaveFile

- 功能

开启音频输入保存文件功能

- 语法

```

MI_S32 MI_AI_SaveFile(MI_AUDIO_DEV AiDevId,    MI_AI_CHN AiChn,
MI_AUDIO_SaveFileInfo_t *pstSaveFileInfo);

```

- 形参

表2-19

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定	输入
pstSaveFileInfo	音频保存文件属性结构体指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: `mi_ai.h`
 - 库文件: `libmi_ai.a/libmi_ai.so`
- 注意
 - 在API版本2.19及以后，已不支持此接口。

2.20. MI_AI_SetVqeVolume

- 功能
设置AI通道的音量大小
- 语法

```
MI_S32 MI_AI_SetVqeVolume(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_S32 s32VolumeDb);
```

- 形参

表2-20

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
s32VolumeDb	AI通道的音量大小	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so
- 注意

下表描述了s32VolumeDb在各个device下对应的增益（dB）。对于Amic和Linein设备，s32VolumeDb表示该设备的模拟增益。对于Macaron系列、Pudding系列、Ispahan系列芯片s32VolumeDb即Dmic所对应的增益。

表2-21

s32VolumeDb	Amic(dB)	Line in(dB)	Pretzel Taiyaki Takoyaki Ikayaki系列 Dmic(dB)
0	-6	-6	0
1	-3	-3	6
2	0	0	12
3	3	3	18
4	6	6	24

5	9	9
6	12	12
7	15	15
8	18	
9	21	
10	24	
11	27	
12	30	
13	33	
14	36	
15	39	
16	42	
17	45	
18	48	
19	51	
20	54	
21	57	

- 举例

下面的代码实现增益设置和获取功能

```
1. MI_S32 ret;
```

```
2. MI_AUDIO_DEV AiDevId = 0;

3. MI_AI_CHN AiChn = 0;

4. MI_AUDIO_Attr_t stAttr;

5. MI_S32 s32VolumeDb;

6. MI_SYS_Init();

7. memset(&stAttr, 0x0, sizeof(MI_AUDIO_Attr_t));

8. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;

9. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;

10. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;

11. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;

12. stAttr.u32PtNumPerFrm = 160;

13. stAttr.u32ChnCnt = 1;

14. /* set public attribute of AI device */

15. ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

16. if (MI_SUCCESS != ret)

17. {

18.     printf("set Dev%d attr err:0x%x\n", AiDevId, ret);

19.     return ret;

20. }

21. /* enable AI device */

22. ret = MI_AI_Enable(AiDevId);

23. if (MI_SUCCESS != ret)

24. {

25.     printf("enable Dev%d err:0x%x\n", AiDevId, ret);

26.     return ret;
```

```
27. }

28. /* enable AI Channel */

29. ret = MI_AI_EnableChn(AiDevId, AiChn);

30. if (MI_SUCCESS != ret)

31. {

32.     printf("enable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

33.     return ret;

34. }

35. /* set channel volume */

36. s32VolumeDb = 12;

37. ret = MI_AI_SetVqeVolume(AiDevId, AiChn, s32VolumeDb);

38. if (MI_SUCCESS != ret)

39. {

40.     printf("set volume Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, r

41.     return ret;

42. }

43. /* get channel volume */

44. ret = MI_AI_GetVqeVolume(AiDevId, AiChn, &s32VolumeDb);

45. if (MI_SUCCESS != ret)

46. {

47.     printf("get volume Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, r
```

```

48.     return ret;

49. }

50. /* disable AI Channel */

51. ret = MI_AI_DisableChn(AiDevId, AiChn);

52. if (MI_SUCCESS != ret)

53. {

54.     printf("disable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret)

55.     return ret;

56. }

57. /* disable AI Device */

58. ret = MI_AI_Disable(AiDevId);

59. if (MI_SUCCESS != ret)

60. {

61.     printf("disable ai %d err:0x%x\n", AiDevId, ret);

62.     return ret;

63. }

64. MI_SYS_Exit();

```

2.21. MI_AI_GetVqeVolume

- 功能

获取AI通道的音量大小

- 语法

```
MI_S32 MI_AI_GetVqeVolume(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_S32 *ps32VolumeDb);
```

- 形参

表2-22

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
ps32VolumeDb	AI通道的音量大小	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so

- 举例

请参考[MI_AI_SetVqeVolume](#) [#220-mi_ai_setvqevolume]举例部分

2.22. MI_AI_SetAencAttr

- 功能

设置AI编码功能相关属性
- 语法

```
MI_S32 MI_AI_SetAencAttr (MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_AI_AencConfig_t *pstAencConfig);
```

• 形参

表2-23

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstAencConfig	音频编码配置结构体指针	输入

• 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

• 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.a/libmi_ai.so libg711.so libg726.so

• 注意

- 需要设置编码功能时，需要先使能AI通道。
- 在API版本2.19及以后，已不支持此接口。

• 举例

下面的代码实现使能和禁用编码参数的功能。

```
1. MI_S32 ret;

2. MI_AUDIO_DEV AiDevId = 0;

3. MI_AI_CHN AiChn = 0;

4. MI_AUDIO_Attr_t stAttr;

5. MI_AI_AencConfig_t stAiAencConfig;

6. MI_SYS_Init();
```

```
7.  memset(&stAttr, 0x0, sizeof(MI_AUDIO_Attr_t));

8.  stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;

9.  stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;

10. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;

11. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;

12. stAttr.u32PtNumPerFrm = 160;

13. stAttr.u32ChnCnt = 1;

14. /* set public attribute of AI device */

15. ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

16. if (MI_SUCCESS != ret)

17. {

18.     printf("set Dev%d attr err:0x%x\n", AiDevId, ret);

19.     return ret;

20. }

21. /* enable AI device */

22. ret = MI_AI_Enable(AiDevId);

23. if (MI_SUCCESS != ret)

24. {

25.     printf("enable Dev%d err:0x%x\n", AiDevId, ret);

26.     return ret;

27. }

28. /* enable AI Channel */

29. ret = MI_AI_EnableChn(AiDevId, AiChn);

30. if (MI_SUCCESS != ret)

31. {
```

```
32.     printf("enable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

33.     return ret;

34. }

35. /* set aenc attr */

36. memset(&stAiAencConfig, 0x0, sizeof(stAiAencConfig));

37. stAiAencConfig.eAencType = E_MI_AUDIO_AENC_TYPE_G711A;

38. ret = MI_AI_SetAencAttr(AiDevId, AiChn, &stAiAencConfig);

39. if (MI_SUCCESS != ret)

40. {

41.     printf("set aenc attr Dev%d Chn%d err:0x%x\n", AiDevId, AiChn

42.     return ret;

43. }

44. /* get aenc attr */

45. memset(&stAiAencConfig, 0x0, sizeof(stAiAencConfig));

46. ret = MI_AI_GetAencAttr(AiDevId, AiChn, &stAiAencConfig);

47. if (MI_SUCCESS != ret)

48. {

49.     printf("get aenc attr Dev%d Chn%d err:0x%x\n", AiDevId, AiChn

50.     return ret;

51. }

52. /* enable aenc */
```



```
53. ret = MI_AI_EnableAenc(AiDevId, AiChn);

54. if (MI_SUCCESS != ret)

55. {

56.     printf("enable aenc Dev%d Chn%d err:0x%x\n", AiDevId, AiChn,

57.         return ret;

58. }

59. /* disable aenc */

60. ret = MI_AI_DisableAenc(AiDevId, AiChn);

61. if (MI_SUCCESS != ret)

62. {

63.     printf("disable aenc Dev%d Chn%d err:0x%x\n", AiDevId, AiChn,

64.         return ret;

65. }

66. /* disable AI Channel */

67. ret = MI_AI_DisableChn(AiDevId, AiChn);

68. if (MI_SUCCESS != ret)

69. {

70.     printf("disable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret)

71.         return ret;

72. }

73. /* disable AI Device */

74. ret = MI_AI_Disable(AiDevId);
```

```
75. if (MI_SUCCESS != ret)
76. {
77.     printf("disable ai %d err:0x%x\n", AiDevId, ret);
78.     return ret;
79. }
80. MI_SYS_Exit();
```

2.23. MI_AI_GetAencAttr

- 功能

获取AI编码功能相关属性

- 语法

```
MI_S32 MI_AI_GetAencAttr (MI_AUDIO_DEV AiDevId,    MI_AI_CHN
AiChn, MI_AI_AencConfig_t *pstAencConfig);
```

- 形参

表2-24

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstAencConfig	音频编码配置结构体指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖
 - 头文件: `mi_ai.h`
 - 库文件: `libmi_ai.a/libmi_ai.so libg711.so libg726.so`
- 注意
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetAencAttr](#) [#222-mi_ai_setaencattr]举例部分。

2.24. MI_AI_EnableAenc

- 功能

使能AI编码功能。
- 语法

```
MI_S32 MI_AI_EnableAenc (MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-25

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so libg711.so libg726.so`
- 注意
 - 在使能编码功能前，必须先使能相应的AI通道和设置编码属性。
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetAencAttr](#) [#222-mi_ai_setaencattr]举例部分。

2.25. MI_AI_DisableAenc

- 功能
禁用AI编码功能。
- 语法

```
MI_S32 MI_AI_DisableAenc (MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-26

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so libg711.so libg726.so`
- 注意
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetAencAttr](#) [#222-mi_ai_setaencattr]举例部分。

2.26. MI_AI_SetAedAttr

- 功能
设置AI声音事件检测功能。
- 语法

```
MI_S32 MI_AI_SetAedAttr(MI_AUDIO_DEV AiDevId,    MI_AI_CHN
AiChn, MI_AI_AedConfig_t *pstAedConfig);
```

- 形参

表2-27

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstAedConfig	声音事件检测配置结构体指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so libAED_LINUX.so`
- 注意
 - 要设置声音事件检测功能参数，需要先使能AI通道。
 - 在API版本2.19及以后，已不支持此接口。
- 举例

下面的代码实现设置声音检测参数、使能声音检测、获取声音检测结果等功能。

```
1. MI_S32 ret;  
2. MI_AUDIO_Attr_t stAttr;  
3. MI_AUDIO_Dev AiDevId = 0;  
4. MI_AI_CHN AiChn = 0;  
5. MI_SYS_ChnPort_t stChnPort;  
6. MI_S32 s32Fd;  
7. fd_set readFdSet;  
8. struct timeval stTimeOut;  
9. MI_AUDIO_Frame_t stAiChFrame;  
10. MI_AUDIO_AecFrame_t stAecFrame;  
11. MI_SYS_ChnPort_t stAiChnOutputPort;  
12. MI_AI_AedConfig_t stAiAedConfig;  
13. MI_AI_AedResult_t stAedResult;  
14. MI_SYS_Init();  
15. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;  
16. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;  
17. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
```

```
18. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;

19. stAttr.u32PtNumPerFrm = 160;

20. stAttr.u32ChnCnt = 1;

21. /* set public attribute of AI device */

22. ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

23. if (MI_SUCCESS != ret)

24. {

25.     printf("set ai %d attr err:0x%x\n", AiDevId, ret);

26.     return ret;

27. }

28. /* get public attribute of AI device */

29. ret = MI_AI_GetPubAttr(AiDevId, &stAttr);

30. if (MI_SUCCESS != ret)

31. {

32.     printf("get ai %d attr err:0x%x\n", AiDevId, ret);

33.     return ret;

34. }

35. /* enable AI device */

36. ret = MI_AI_Enable(AiDevId);

37. if (MI_SUCCESS != ret)

38. {

39.     printf("enable ai %d err:0x%x\n", AiDevId, ret);

40.     return ret;

41. }

42. /* enable AI Channel */
```

```
43. ret = MI_AI_EnableChn(AiDevId, AiChn);

44. if (MI_SUCCESS != ret)

45. {

46.     printf("enable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

47.     return ret;

48. }

49. memset(&stAiAedConfig, 0x0, sizeof(stAiAedConfig));

50. stAiAedConfig.bEnableNr = TRUE;

51. stAiAedConfig.eSensitivity = E_MI_AUDIO_AED_SEN_HIGH;

52. stAiAedConfig.s32OperatingPoint = -5;

53. stAiAedConfig.s32VadThresholdDb = -40;

54. stAiAedConfig.s32LsdThresholdDb = -15;

55. /* set aed attr */

56. ret = MI_AI_SetAedAttr(AiDevId, AiChn, &stAiAedConfig);

57. if (MI_SUCCESS != ret)

58. {

59.     printf("set aed attr Dev%d Chn%d err:0x%x\n", AiDevId, AiChn,

60.         return ret;

61. }

62. /* get aed attr */

63. ret = MI_AI_GetAedAttr(AiDevId, AiChn, &stAiAedConfig);

64. if (MI_SUCCESS != ret)
```



```
65. {
66.     printf("get aed attr Dev%d Chn%d err:0x%x\n", AiDevId, AiChn,
67.         return ret;
68. }
69. /* enable aed */
70. ret = MI_AI_EnableAed(AiDevId, AiChn);
71. if (MI_SUCCESS != ret)
72. {
73.     printf("enable aed Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, r
74.         return ret;
75. }
76. /* set buffer depth */
77. stAiChnOutputPort.eModId = E_MI_MODULE_ID_AI;
78. stAiChnOutputPort.u32DevId = AiDevId;
79. stAiChnOutputPort.u32ChnId = AiChn;
80. stAiChnOutputPortu32PortId = 0;
81. MI_SYS_SetChnOutputPortDepth(&stAiChnOutputPort, 1, 8);
82. /* get port fd */
83. stChnPort.eModId = E_MI_MODULE_ID_AI;
84. stChnPort.u32DevId = AiDevId;
85. stChnPort.u32ChnId = AiChn;
86. stChnPort.u32PortId = 0;
87. ret = MI_SYS_GetFd(&stChnPort, &s32Fd);
```

```

88. if (MI_SUCCESS != ret)
89. {
90.     printf("Dev%d Chn%d failed to call MI_SYS_GetFd!!!\n", AiDevId, AiChn);

91.     return ret;
92. }

93. /* select 100ms */
94. FD_ZERO(&readFdSet);
95. FD_SET(s32Fd, &readFdSet);
96. stTimeOut.tv_sec = 0;
97. stTimeOut.tv_usec = 100 * 1000;

98.
ret = select(s32Fd + 1, &readFdSet, NULL, NULL, &stTimeOut);

99. if (FD_ISSET(s32Fd, &readFdSet))
100. {
101.     ret = MI_AI_GetFrame(AiDevId, AiChn, &stAiChFrame, &stAecFrame);

102.     if (MI_SUCCESS == ret)
103.     {
104.         /* get aed result */
105.         MI_AI_GetAedResult(AiDevId, AiChn, &stAedResult);
106.         MI_AI_ReleaseFrame(AiDevId, AiChn, &stAiChFrame, &stAecFrame);

107.     }
108. }

```

```
109. /* disable aed */
110. ret = MI_AI_DisableAed(AiDevId, AiChn);
111. if (MI_SUCCESS != ret)
112. {
113.     printf("disable aed Dev%d Chn%d err:0x%x\n", AiDevId, AiChn,
114.         return ret;
115. }
116. /* disable AI Channel */
117. ret = MI_AI_DisableChn(AiDevId, AiChn);
118. if (MI_SUCCESS != ret)
119. {
120.     printf("disable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret)
121.     return ret;
122. }
123. /* disable AI Device */
124. ret = MI_AI_Disable(AiDevId);
125. if (MI_SUCCESS != ret)
126. {
127.     printf("disable ai %d err:0x%x\n", AiDevId, ret);
128.     return ret;
129. }
130. MI_SYS_Exit();
```

2.27. MI_AI_GetAedAttr

- 功能

获取AI声音事件检测功能配置。

- 语法

```
MI_S32 MI_AI_GetAedAttr(MI_AUDIO_DEV AiDevId,MI_AI_CHN AiChn,
MI_AI_AedConfig_t *pstAedConfig);
```

- 形参

表2-28

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstAedConfig	声音事件检测配置结构体指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.a/libmi_ai.so libAED_LINUX.so

- 注意

- 在API版本2.19及以后，已不支持此接口。

- 举例

请参考[MI_AI_SetAedAttr](#) [#226-mi_ai_setaedattr]举例部分

2.28. MI_AI_EnableAed

- 功能

使能AI声音事件检测功能。

- 语法

```
MI_S32 MI_AI_EnableAed(MI_AUDIO_DEV AiDevId,    MI_AI_CHN
AiChn);
```

- 形参

表2-29

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so libAED_LINUX.so`

- 注意

- 在使能声音事件检测功能前，必须先使能AI通道和设置声音事件检测属性。
- 在API版本2.19及以后，已不支持此接口。

- 举例

请参考[MI_AI_SetAedAttr](#) [#326-mi_ai_setaedattr]举例部分

2.29. MI_AI_DisableAed

- 功能
禁止AI声音事件检测功能。
- 语法

```
MI_S32 MI_AI_DisableAed(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-30

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so libAED_LINUX.so
- 注意
 - 在API版本2.19及以后，已不支持此接口。
- 举例
请参考[MI_AI_SetAedAttr](#) [#326-mi_ai_setaedattr]举例部分

2.30. MI_AI_GetAedResult

• 功能

获取AI声音事件检测结果。

• 语法

```
MI_S32 MI_AI_GetAedResult(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_AI_AedResult_t *pstAedResult);
```

• 形参

表2-31

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstAedResult	声音事件检测结果结构体指针	输出

• 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

• 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.a/libmi_ai.so libAED_LINUX.so

• 注意

- 在API版本2.19及以后，已不支持此接口。

• 举例

请参考[MI_AI_SetAedAttr](#) [#226-mi_ai_setaedattr]举例部分

2.31. MI_AI_SetExtAecChn

• 功能

设置回声消除功能参考的AI通道。

• 语法

```
MI_S32 MI_AI_SetExtAecChn(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_AI_CHN AiAECSndChn);
```

• 形参

表2-32

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
AiAECSndChn	参考的AI通道号	输入

• 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

• 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.a/libmi_ai.so

• 注意

设置外部回声参考通道时，需要在通道使能前进行设置，目前仅支持在Mono模式下调用此接口。

• 举例

下面的代码进行外部AEC参考通道的设置。

```
1. MI_S32 ret;  
  
2. MI_AUDIO_Attr_t stAttr;
```



```
3.  MI_AUDIO_Dev AiDevId = 0;

4.  MI_AI_CHN AiChn = 0;

5.  MI_AI_CHN ExtAecChn = 0;

6.  MI_SYS_Init();

7.  stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;

8.  stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;

9.  stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;

10. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;

11. stAttr.u32PtNumPerFrm = 160;

12. stAttr.u32ChnCnt = 1;

13. /* set public attribute of AI device */

14. ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

15. if (MI_SUCCESS != ret)

16. {

17.     printf("set ai %d attr err:0x%x\n", AiDevId, ret);

18.     return ret;

19. }

20. /* get public attribute of AI device */

21. ret = MI_AI_GetPubAttr(AiDevId, &stAttr);

22. if (MI_SUCCESS != ret)

23. {

24.     printf("get ai %d attr err:0x%x\n", AiDevId, ret);

25.     return ret;

26. }
```

```
27. /* enable AI device */

28. ret = MI_AI_Enable(AiDevId);

29. if (MI_SUCCESS != ret)

30. {

31.     printf("enable ai %d err:0x%x\n", AiDevId, ret);

32.     return ret;

33. }

34. /* set ext Aec Chn */

35. ret = MI_AI_SetExtAecChn(AiDevId, AiChn, ExtAecChn);

36. if (MI_SUCCESS != ret)

37. {

38.     printf("set ext aec chn Dev%d Chn%d err:0x%x\n", AiDevId, AiC

39.     return ret;

40. }

41. /* enable AI Channel */

42. ret = MI_AI_EnableChn(AiDevId, AiChn);

43. if (MI_SUCCESS != ret)

44. {

45.     printf("enable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);

46.     return ret;

47. }

48. /* disable AI Channel */

49. ret = MI_AI_DisableChn(AiDevId, AiChn);
```

```

50. if (MI_SUCCESS != ret)
51. {
52.     printf("disable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret)

53.     return ret;
54. }

55. /* disable AI Device */
56. ret = MI_AI_Disable(AiDevId);
57. if (MI_SUCCESS != ret)
58. {
59.     printf("disable ai %d err:0x%x\n", AiDevId, ret);
60.     return ret;
61. }
62. MI_SYS_Exit();

```

2.32. MI_AI_SetSslInitAttr

- 功能

设置声源定位功能的初始化参数。

- 语法

```

MI_S32 MI_AI_SetSslInitAttr(MI_AUDIO_DEV AiDevId,    MI_AI_CHN
AiChn, MI_AI_SslInitAttr_t *    pstSslInitAttr);

```

- 形参

表2-33

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstSslInitAttr	声源定位功能初始化结构体指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so libSSL_LINUX.so
- 注意
 - 声源定位的初始化参数只能在使能前进行设置，使能后不能再进行设置。
 - 在API版本2.19及以后，已不支持此接口。
- 举例

下面的代码实现声源定位参数的设置，使能声源定位和获取声源定位结果。

```
1. MI_S32 ret;  
2. MI_AUDIO_Attr_t stAttr;  
3. MI_AUDIO_Dev AiDevId = 0;  
4. MI_AI_CHN AiChn = 0;  
5. MI_SYS_ChnPort_t stChnPort;  
6. MI_S32 s32Fd;  
7. fd_set readFdSet;  
8. struct timeval stTimeOut;
```

```
9. MI_AUDIO_Frame_t stAiChFrame;

10. MI_AUDIO_AecFrame_t stAecFrame;

11. MI_SYS_ChnPort_t stAiChnOutputPort;

12. MI_S32 s32Doa;

13. MI_AI_SslInitAttr_t stSslInit = {

14.     .bBfMode = FALSE,

15.     .u32MicDistance = 3,

16. };

17. MI_AI_SslConfigAttr_t stSslConfig = {

18.     .s32Temperature = 25,

19.     .s32NoiseGateDbfs = -40,

20.     .s32DirectionFrameNum = 300,

21. };

22. MI_AI_SslInitAttr_t stAiGetSslInitAttr;

23. MI_AI_SslConfigAttr_t stAiGetSslConfigAttr;

24. MI_SYS_Init();

25. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;

26. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;

27. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_STEREO;

28. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;

29. stAttr.u32PtNumPerFrm = 160;

30. stAttr.u32ChnCnt = 1;

31. /* set public attribute of AI device */

32. ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

33. if (MI_SUCCESS != ret)
```

```
34. {  
  
35.     printf("set ai %d attr err:0x%x\n", AiDevId, ret);  
  
36.     return ret;  
  
37. }  
  
38. /* get public attribute of AI device */  
  
39. ret = MI_AI_GetPubAttr(AiDevId, &stAttr);  
  
40. if (MI_SUCCESS != ret)  
  
41. {  
  
42.     printf("get ai %d attr err:0x%x\n", AiDevId, ret);  
  
43.     return ret;  
  
44. }  
  
45. /* enable AI device */  
  
46. ret = MI_AI_Enable(AiDevId);  
  
47. if (MI_SUCCESS != ret)  
  
48. {  
  
49.     printf("enable ai %d err:0x%x\n", AiDevId, ret);  
  
50.     return ret;  
  
51. }  
  
52. /* enable AI Channel */  
  
53. ret = MI_AI_EnableChn(AiDevId, AiChn);  
  
54. if (MI_SUCCESS != ret)  
  
55. {  
  
56.     printf("enable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);
```

```
57.     return ret;

58. }

59. /* set ssl init attr */

60. ret = MI_AI_SetSslInitAttr(AiDevId, AiChn, &stSslInit);

61. if (MI_SUCCESS != ret)

62. {

63.     printf("set ssl init attr Dev%d Chn%d err:0x%x\n", AiDevId, A

64.     return ret;

65. }

66. /* get ssl init attr */

67.
ret = MI_AI_GetSslInitAttr(AiDevId, AiChn, &stAiGetSslInitAttr);

68. if (MI_SUCCESS != ret)

69. {

70.     printf("get ssl init attr Dev%d Chn%d err:0x%x\n", AiDevId, A

71.     return ret;

72. }

73. /* set ssl config attr */

74. ret = MI_AI_SetSslConfigAttr(AiDevId, AiChn, &stSslConfig);

75. if (MI_SUCCESS != ret)

76. {

77.     printf("set ssl config attr Dev%d Chn%d err:0x%x\n", AiDevId,
```

```
78.     return ret;

79. }

80. /* get ssl config attr */

81.
ret = MI_AI_GetSslConfigAttr(AiDevId, AiChn, &stAiGetSslConfigAtt

82. if (MI_SUCCESS != ret)

83. {

84.
    printf("get ssl config attr Dev%d Chn%d err:0x%x\n", AiDevId,

85.     return ret;

86. }

87. /* enable ssl */

88. ret = MI_AI_EnableSsl(AiDevId, AiChn);

89. **if** (MI_SUCCESS != ret)

90. {

91.
    printf("enable ssl Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, r

92.     return ret;

93. }

94. /* set buffer depth */

95. stAiChnOutputPort.eModId = E_MI_MODULE_ID_AI;

96. stAiChnOutputPort.u32DevId = AiDevId;

97. stAiChnOutputPort.u32ChnId = AiChn;

98. stAiChnOutputPortu32PortId = 0;
```



```
99. MI_SYS_SetChnOutputPortDepth(&stAiChnOutputPort, 1, 8);

100. /* get port fd */

101. stChnPort.eModId = E_MI_MODULE_ID_AI;

102. stChnPort.u32DevId = AiDevId;

103. stChnPort.u32ChnId = AiChn;

104. stChnPort.u32PortId = 0;

105. ret = MI_SYS_GetFd(&stChnPort, &s32Fd);

106. if (MI_SUCCESS != ret)

107. {

108.     printf("Dev%d Chn%d failed to call MI_SYS_GetFd!!!\n", AiDevId, AiChn);

109.     return ret;

110. }

111. /* select 100ms */

112. FD_ZERO(&readFdSet);

113. FD_SET(s32Fd, &readFdSet);

114. stTimeOut.tv_sec = 0;

115. stTimeOut.tv_usec = 100 * 1000;

116. ret = select(s32Fd + 1, &readFdSet, NULL, NULL, &stTimeOut);

117. if (FD_ISSET(s32Fd, &readFdSet))

118. {

119.     ret = MI_AI_GetFrame(AiDevId, AiChn, &stAiChFrame, &stAecFrame);

120.     if (MI_SUCCESS == ret)
```

```
121.     {
122.         /* get ssl result */
123.         MI_AI_GetSslDoa(AiDevId, AiChn, &s32Doa);
124.         MI_AI_ReleaseFrame(AiDevId, AiChn, &stAiChFrame, &stAecFr
125.     }
126. }
127. /* disable ssl */
128. ret = MI_AI_DiableSsl(AiDevId, AiChn);
129. if (MI_SUCCESS != ret)
130. {
131.     printf("disable ssl Dev%d Chn%d err:0x%x\n", AiDevId, AiChn,
132.     return ret;
133. }
134. /* disable AI Channel */
135. ret = MI_AI_DisableChn(AiDevId, AiChn);
136. if (MI_SUCCESS != ret)
137. {
138.     printf("disable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret)
139.     return ret;
140. }
141. /* disable AI Device */
142. ret = MI_AI_Disable(AiDevId);
```

```

143. if (MI_SUCCESS != ret)
144. {
145.     printf("disable ai %d err:0x%x\n", AiDevId, ret);
146.     return ret;
147. }
148. MI_SYS_Exit();

```

2.33. MI_AI_GetSslInitAttr

- 功能

获取声源定位功能的初始化参数。

- 语法

```

MI_S32 MI_AI_GetSslInitAttr(MI_AUDIO_DEV AiDevId,    MI_AI_CHN
AiChn,MI_AI_SslInitAttr_t * pstSslInitAttr);

```

- 形参

表2-34

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstSslInitAttr	声源定位功能初始化结构体指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖
 - 头文件: `mi_ai.h`
 - 库文件: `libmi_ai.a/libmi_ai.so libSSL_LINUX.so`
- 注意
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetSslInitAttr](#) [#232-mi_ai_setsslinitattr]举例部分

2.34. MI_AI_SetSslConfigAttr

- 功能
设置声源定位功能的配置参数。
- 语法

```
MI_S32 MI_AI_SetSslConfigAttr(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_AI_SslConfigAttr_t * pstSslConfigAttr);
```

- 形参

表2-35

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstSslConfigAttr	声源定位功能配置结构体指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。

- 依赖
 - 头文件: `mi_ai.h`
 - 库文件: `libmi_ai.a/libmi_ai.so libSSL_LINUX.so`
- 注意
 - 可以对每一帧数据进行设置。
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetSslInitAttr](#) [#232-mi_ai_setsslinitattr]举例部分

2.35. MI_AI_GetSslConfigAttr

- 功能
获取声源定位功能的配置参数。
- 语法

```
MI_S32 MI_AI_GetSslConfigAttr(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_AI_SslConfigAttr_t* pstSslConfigAttr);
```

- 形参

表2-36

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstSslConfigAttr	声源定位功能配置结构体指针	输出

- 返回值
 - 0 成功。

- 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.a/libmi_ai.so libSSL_LINUX.so
- 注意
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetSslInitAttr](#) [#232-mi_ai_setsslinitattr]举例部分

2.36. MI_AI_EnableSsl

- 功能
使能声源定位功能。
- 语法

```
MI_S32 MI_AI_EnableSsl(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-37

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so libSSL_LINUX.so`
- 注意
 - 在使能声源定位功能前，必须先使能AI通道以及设置声源定位的初始化参数和配置参数。
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetSslInitAttr](#) [[#232mi_ai_setsslinitattr](#)]举例部分

2.37. MI_AI_DisableSsl

- 功能
禁止声源定位功能。
- 语法

```
MI_S32 MI_AI_DisableSsl(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-38

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [[#jump](#)]。
- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so libSSL_LINUX.so`
- 注意
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetSslInitAttr](#) [#232-mi_ai_setsslinitattr]举例部分

2.38. MI_AI_GetSslDoa

- 功能
获取声源定位功能的检测结果。
- 语法

```
MI_S32 MI_AI_GetSslDoa(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_S32 *ps32SslDoa);
```

- 形参

表2-39

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
ps32SslDoa	声源定位功能的检测结果	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.a/libmi_ai.so libSSL_LINUX.so`
- 注意
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetSslInitAttr](#) [#232-mi_ai_setsslinitattr]举例部分

2.39. MI_AI_SetBfInitAttr

- 功能
设置波束成形功能的初始化参数。
- 语法

```
MI_S32 MI_AI_SetBfInitAttr(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_AI_BfInitAttr_t* pstBfInitAttr);
```

- 形参

表2-40

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstBfInitAttr	波束成形功能初始化结构体指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.so/libmi_ai.a libBF_LINUX.so
- 注意
 - 波束成形初始化参数只能在使能前进行设置。
 - 在API版本2.19及以后，已不支持此接口。
- 举例

下面的代码实现波束成形功能的参数设置和使能。

```
1. MI_S32 ret;  
2. MI_AUDIO_Attr_t stAttr;  
3. MI_AUDIO_Dev AiDevId = 0;  
4. MI_AI_CHN AiChn = 0;  
5. MI_AI_BfInitAttr_t stBfInit = {  
6.     .u32ChanCnt = 2,  
7.     .u32MicDistance = 3,  
8. };  
9. MI_AI_BfConfigAttr_t stBfConfig = {  
10.     .s32Temperature = 25,  
11.     .s32NoiseGateDbfs = -40,  
12.     .s32NoiseSupressionMode = 8,  
13.     .s32NoiseEstimation = 1,  
14.     .outputGain = 0.7,  
15. };  
16. MI_AI_BfInitAttr_t stAiGetBfInitAttr;  
17. MI_AI_BfConfigAttr_t stAiGetBfConfigAttr;  
18. MI_BOOL bAiSetBfDoa = TRUE;
```

```
19. MI_S32 s32AiBfDoa = 0;

20. MI_SYS_Init();

21. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;

22. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;

23. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_STEREO;

24. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;

25. stAttr.u32PtNumPerFrm = 160;

26. stAttr.u32ChnCnt = 1;

27. /* set public attribute of AI device */

28. ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

29. if (MI_SUCCESS != ret)

30. {

31.     printf("set ai %d attr err:0x%x\n", AiDevId, ret);

32.     return ret;

33. }

34. /* get public attribute of AI device */

35. ret = MI_AI_GetPubAttr(AiDevId, &stAttr);

36. if (MI_SUCCESS != ret)

37. {

38.     printf("get ai %d attr err:0x%x\n", AiDevId, ret);

39.     return ret;

40. }

41. /* enable AI device */

42. ret = MI_AI_Enable(AiDevId);
```

```
43. if (MI_SUCCESS != ret)
44. {
45.     printf("enable ai %d err:0x%x\n", AiDevId, ret);
46.     return ret;
47. }
48. /* enable AI Channel */
49. ret = MI_AI_EnableChn(AiDevId, AiChn);
50. if (MI_SUCCESS != ret)
51. {
52.     printf("enable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);
53.     return ret;
54. }
55. /* set bf init attr */
56. ret = MI_AI_SetBfInitAttr(AiDevId, AiChn, &stBfInit);
57. if (MI_SUCCESS != ret)
58. {
59.     printf("set bf init attr Dev%d Chn%d err:0x%x\n", AiDevId, Ai
60.     return ret;
61. }
62. /* get bf init attr */
63. ret = MI_AI_GetBfInitAttr(AiDevId, AiChn, &stAiGetBfInitAttr);
64. if (MI_SUCCESS != ret)
```

```
65. {
66.     printf("get bf init attr Dev%d Chn%d err:0x%x\n", AiDevId, Ai
67.         return ret;
68. }
69. /* set bf config attr */
70. ret = MI_AI_SetBfConfigAttr(AiDevId, AiChn, &stBfConfig);
71. if (MI_SUCCESS != ret)
72. {
73.     printf("set bf config attr Dev%d Chn%d err:0x%x\n", AiDevId,
74.         return ret;
75. }
76. /* get bf config attr */
77. ret = MI_AI_GetBfConfigAttr(AiDevId, AiChn, &stAiGetBfConfigAttr)
78. if (MI_SUCCESS != ret)
79. {
80.     printf("get bf config attr Dev%d Chn%d err:0x%x\n", AiDevId,
81.         return ret;
82. }
83. /* set bf angle */
84. if (bAiSetBfDoa)
85. {
```

```
86.     ret = MI_AI_SetBfAngle(AiDevId, AiChn, s32AiBfDoa);
87.     if (MI_SUCCESS != ret)
88.     {
89.
90.         printf("set bf doa Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);
91.
92.         return ret;
93.     }
94. }
95.
96. /* enable bf */
97. ret = MI_AI_EnableBf(AiDevId, AiChn);
98. if (MI_SUCCESS != ret)
99. {
100.
101.     printf("enable bf Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);
102.
103.     return ret;
104. }
105.
106. /* do something */
107. /* enable bf */
108. ret = MI_AI_DisableBf(AiDevId, AiChn);
109. if (MI_SUCCESS != ret)
110. {
111.
112.     printf("disable bf Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);
113.
114.     return ret;
115. }
```

```

107. }

108. /* disable AI Channel */

109. ret = MI_AI_DisableChn(AiDevId, AiChn);

110. if (MI_SUCCESS != ret)

111. {

112.     printf("disable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret)

113.     return ret;

114. }

115. /* disable AI Device */

116. ret = MI_AI_Disable(AiDevId);

117. if (MI_SUCCESS != ret)

118. {

119.     printf("disable ai %d err:0x%x\n", AiDevId, ret);

120.     return ret;

121. }

122. MI_SYS_Exit();

```

2.40. MI_AI_GetBfInitAttr

- 功能

获取波束成形功能的初始化参数。

- 语法

```

MI_S32 MI_AI_GetBfInitAttr(MI_AUDIO_DEV AiDevId, MI_AI_CHN
AiChn, MI_AI_BfInitAttr_t *pstBfInitAttr);

```

- 形参

表2-41

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstBfInitAttr	波束成形功能初始化结构体指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.so/libmi_ai.a libBF_LINUX.so`

- 注意

- 在API版本2.19及以后，已不支持此接口。

- 举例

请参考[MI_AI_SetBfInitAttr](#) [#239-mi_ai_setbfinitattr]举例部分。

2.41. MI_AI_SetBfConfigAttr

- 功能

设置波束成形功能的配置参数。

- 语法

```
MI_S32 MI_AI_SetBfConfigAttr(MI_AUDIO_DEV AiDevId,MI_AI_CHN AiChn, MI_AI_BfConfigAttr_t* pstBfConfigAttr);
```


- 形参

表2-42

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstBfInitAttr	波束成形功能配置结构体指针	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.so/libmi_ai.a libBF_LINUX.so`

- 注意

- 在API版本2.19及以后，已不支持此接口。

- 举例

请参考[MI_AI_SetBfInitAttr](#) [#239-mi_ai_setbfinitattr]举例部分。

2.42. MI_AI_GetBfConfigAttr

- 功能

获取波束成形功能的配置参数。

- 语法

```
MI_S32 MI_AI_GetBfConfigAttr(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_AI_BfConfigAttr_t* pstBfConfigAttr);
```

- 形参

表2-43

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pstBfConfigAttr	波束成形功能配置结构体指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: `mi_ai.h`
- 库文件: `libmi_ai.so/libmi_ai.a libBF_LINUX.so`

- 注意

- 在API版本2.19及以后，已不支持此接口。

- 举例

请参考[MI_AI_SetBfInitAttr](#) [#239-mi_ai_setbfinitattr]举例部分。

2.43. MI_AI_EnableBf

- 功能

使能波束成形功能。

- 语法

```
MI_S32 MI_AI_EnableBf(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-44

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.so/libmi_ai.a libBF_LINUX.so
- 注意
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetBfInitAttr](#) [#239-mi_ai_setbfinitattr]举例部分。

2.44. MI_AI_DisableBf

- 功能

禁止波束成形功能。
- 语法

```
MI_S32 MI_AI_DisableBf(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn);
```

- 形参

表2-45

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: `mi_ai.h`
 - 库文件: `libmi_ai.so/libmi_ai.a libBF_LINUX.so`
- 注意
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetBfInitAttr](#) [#239-mi_ai_setbfinitattr]举例部分。

2.45. MI_AI_SetBfAngle

- 功能
设置波束成形功能的作用角度。
- 语法

```
MI_S32 MI_AI_SetBfAngle(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn,  
MI_S32 s32BfAngle);
```

- 形参

表2-46

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
s32BfAngle	角度	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.so/libmi_ai.a libBF_LINUX.so
- 注意
 - 在API版本2.19及以后，已不支持此接口。
- 举例

请参考[MI_AI_SetBfInitAttr](#) [#239-mi_ai_setbfinitattr]举例部分。

2.46. MI_AI_SetMute

- 功能

设置AI通道的静音参数。
- 语法

```
MI_S32 MI_AI_SetMute(MI_AUDIO_DEV AiDevId,    MI_AI_CHN AiChn,
MI_BOOL bMute);
```

- 形参

表2-47

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
bMute	静音参数	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.so/libmi_ai.a
- 举例

请下面的代码实现设置和获取AI通道的静音状态。

```
1. MI_S32 ret;  
2. MI_AUDIO_Attr_t stAttr;  
3. MI_AUDIO_Dev AiDevId = 0;  
4. MI_AI_CHN AiChn = 0;  
5. MI_BOOL bMute = TRUE;  
6. MI_SYS_Init();  
7. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;  
8. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;  
9. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_STEREO;  
10. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_SLAVE;  
11. stAttr.u32PtNumPerFrm = 160;
```

```
12. stAttr.u32ChnCnt = 1;

13. /* set public attribute of AI device */

14. ret = MI_AI_SetPubAttr(AiDevId, &stAttr);

15. if (MI_SUCCESS != ret)

16. {

    17.     printf("set ai %d attr err:0x%x\n", AiDevId, ret);

    18.     return ret;

    19. }

    20. /* get public attribute of AI device */

    21. ret = MI_AI_GetPubAttr(AiDevId, &stAttr);

    22. if (MI_SUCCESS != ret)

    23. {

        24.     printf("get ai %d attr err:0x%x\n", AiDevId, ret);

        25.     return ret;

        26. }

27. /* enable AI device */

28. ret = MI_AI_Enable(AiDevId);

29. if (MI_SUCCESS != ret)

30. {

    31.     printf("enable ai %d err:0x%x\n", AiDevId, ret);

    32.     return ret;

    33. }

34. /* enable AI Channel */

35. ret = MI_AI_EnableChn(AiDevId, AiChn);
```

```
36. if (MI_SUCCESS != ret)
37. {
38.     printf("enable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret);
39.     return ret;
40. }
41. /* set mute status */
42. ret = MI_AI_SetMute(AiDevId, AiChn, bMute);
43. if (MI_SUCCESS != ret)
44. {
45.     printf("set mute status Dev%d Chn%d err:0x%x\n", AiDevId, AiC
46.     return ret;
47. }
48. /* get mute status */
49. ret = MI_AI_GetMute(AiDevId, AiChn, &bMute);
50. if (MI_SUCCESS != ret)
51. {
52.     printf("get mute status Dev%d Chn%d err:0x%x\n", AiDevId, AiC
53.     return ret;
54. }
55. /* disable AI Channel */
56. ret = MI_AI_DisableChn(AiDevId, AiChn);
57. if (MI_SUCCESS != ret)
```



```

58. {
59.     printf("disable Dev%d Chn%d err:0x%x\n", AiDevId, AiChn, ret)

60.     return ret;
61. }

62. /* disable AI Device */
63. ret = MI_AI_Disable(AiDevId);
64. if (MI_SUCCESS != ret)
65. {
66.     printf("disable ai %d err:0x%x\n", AiDevId, ret);
67.     return ret;
68. }

69. MI_SYS_Exit();

```

2.47. MI_AI_GetMute

- 功能

获取AI通道的静音参数。

- 语法

```

MI_S32 MI_AI_GetMute(MI_AUDIO_DEV AiDevId,    MI_AI_CHN AiChn,
MI_BOOL *pbMute);

```

- 形参

表2-48

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入
pbMute	静音参数	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.so/libmi_ai.a
- 举例

请参考[MI_AI_SetMute](#) [#246-mi_ai_setmute]举例部分。

2.48. MI_AI_InitDev

- 功能
初始化AI设备。
- 语法

```
MI_S32 MI_AI_InitDev(MI_AI_InitParam_t *pstInitParam);
```

- 形参

参数名称	描述	输入/输出
pstInitParam	设备初始化参数	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
 - 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.so/libmi_ai.a
 - 注意
 - 本接口必须和MI_AI_DeInitDev成对使用，不可单独重复调用，否则返回失败。
 - 仅用于STR状态唤醒后重新初始化AI模块。
-

2.49. MI_AI_DeInitDev

- 功能
反初始化AI设备。
- 语法

```
MI_S32 MI_AI_DeInitDev(void);
```

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.so/libmi_ai.a
- 注意
 - 此函数必须在初始化设备后调用，否则返回失败。
 - 如果本接口在app退出前没有调用，内部会自动反初始化设备。

- 本接口必须和MI_AI_InitDev成对使用，不可单独重复调用，否则返回失败。

2.50. MI_AI_DupChn

- 功能
同步AI通道的状态。
- 语法

```
MI_S32 MI_AI_DupChn(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn)
```

- 形参

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。支持的通道范围由AI设备属性中的最大通道个数u32ChnCnt决定。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.so/libmi_ai.a

- 注意
此接口仅适用于dual os环境。在rtos环境下已初始化AI通道，切换到Linux环境时，用于同步AI通道的状态。

2.51. MI_AI_DevIsEnable

- 功能

获取AI设备的使能状态。

- 语法

```
MI_S32 MI_AI_DevIsEnable(MI_AUDIO_DEV AiDevId, MI_BOOL *pbEnable)
```

- 形参

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
pbEnable	音频设备使能状态。	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [jump]。

- 依赖

- 头文件: mi_ai.h
- 库文件: libmi_ai.so/libmi_ai.a

2.52. MI_AI_ChnlIsEnable

- 功能

获取AI通道的使能状态。

- 语法

```
MI_S32 MI_AI_ChnlIsEnable(MI_AUDIO_DEV AiDevId, MI_AI_CHN AiChn, MI_BOOL *pbEnable)
```

- 形参

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号。 支持的通道范围由AI设备属性中的最大通道个数 u32ChnCnt决定。	输入
pbEnable	音频通道使能状态。	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ai.h
 - 库文件: libmi_ai.so/libmi_ai.a

3. AI 数据类型

3.1. AI模块相关数据类型定义

表3-1

数据类型	定义
MI_AUDIO_DEV [#32-mi_audio_dev]	定义音频输入/输出设备编号
MI_AUDIO_MAX_CHN_NUM [#33-mi_audio_max_chn_num]	定义音频输入/输出设备的最大通道数
MI_AI_CHN [#34-mi_ai_chn]	定义音频输入通道
数据类型	定义
MI_AUDIO_SampleRate_e [#35-mi_audio_sample_rate_e]	定义音频采样率

MI_AUDIO_Samplerate_e [#35-mi_audio_samplerate_e]	定义音频采样率
MI_AUDIO_Bitwidth_e [#36-mi_audio_bitwidth_e]	定义音频采样精度
MI_AUDIO_Mode_e [#37-mi_audio_mode_e]	定义音频输入输出工作模式
MI_AUDIO_SoundMode_e [#38-mi_audio_soundmode_e]	定义音频声道模式
MI_AUDIO_AencType_e [#39-mi_audio_aenctype_e]	定义音频编码类型
MI_AUDIO_G726Mode_e [#310-mi_audio_g726mode_e]	定义G726工作模式
MI_AUDIO_I2sFmt_e [#311-mi_audio_i2sfmt_e]	I2S 格式设定
MI_AUDIO_I2sMclk_e [#312-mi_audio_i2smclk_e]	I2S MCLK 设定
MI_AUDIO_I2sConfig_t [#313-mi_audio_i2sconfig_t]	定义I2S属性结构体
MI_AUDIO_Attr_t [#314-mi_audio_attr_t]	定义音频输入输出设备属性结构体
MI_AI_ChnParam_t [#315-mi_ai_chnparam_t]	定义通道参数结构体
MI_AUDIO_Frame_t [#316-mi_audio_frame_t]	定义音频帧数据结构体
MI_AUDIO_AecFrame_t [#317-mi_audio_aecframe_t]	定义回声抵消参考帧信息结构体
MI_AUDIO_SaveFileInfo_t [#318-mi_audio_savefileinfo_t]	定义音频保存文件功能配置信息结构体
MI_AI_VqeConfig_t [#319-mi_ai_vqeconfig_t]	定义音频输入声音质量增强配置信息结构体
MI_AUDIO_HpfConfig_t [#320-mi_audio_hpfconfig_t]	定义音频高通滤波功能配置信息结构体
MI_AUDIO_HpfFreq_e [#321-mi_audio_hpffreq_e]	定义音频高通滤波截止频率
数据类型	

MI_AI_AecConfig_t [#322-mi_ai_aecconfig_t]	定义音频回声抵消配置信息结构体
MI_AUDIO_AnrcConfig_t [#323-mi_audio_anrconfig_t]	定义音频语音降噪功能配置信息结构体
MI_AUDIO_NrSpeed_e [#324-mi_audio_nrspeed_e]	定义噪声收敛速度
MI_AUDIO_AgcConfig_t [#325-mi_audio_agcconfig_t]	定义音频自动增益控制配置信息结构体
MI_AUDIO_EqConfig_t [#327-mi_audio_eqconfig_t]	定义音频均衡器功能配置信息结构体
MI_AI_AencGConfig_t [#328-mi_ai_aencgconfig_t]	定义音频编码功能配置信息结构体
MI_AUDIO_AencG711Config_t [#329-mi_audio_aencg711config_t]	定义音频编码功能配置信息结构体
MI_AUDIO_AencG726Config_t [#330-mi_audio_aencg726config_t]	定义音频编码功能配置信息结构体
MI_AUDIO_AlgorithmMode_e [#331-mi_audio_algorithmmode_e]	定义音频算法的运行模式
MI_AI_AedConfig_t [#332-mi_ai_aedconfig_t]	定义声音事件检测功能配置信息结构体
[MI_AUDIO_AedSensitivity_e/MI_AUDIO_BabyCrySensitivity_e] (#333-mi_audio_aedsensitivity_e/mi_audio_babycrysensitivity_e)	
)	定义声音事件检测的灵敏度
MI_AI_AedResult_t [#334-mi_ai_aedresult_t]	定义声音事件检测的结果
MI_AI_ChngainConfig_t [#335-mi_ai_chngainconfig_t]	定义音频通道增益设置结构体
数据类型 MI_AI_SslinitAttr_t [#336-mi_ai_sslinitattr_t]	定义 定义音频声源定位功

	能的初始化信息结构体
MI_AI_SslConfigAttr_t [#337-mi_ai_sslconfigattr_t]	定义音频声源定位功能的配置信息结构体
MI_AI_BfInitAttr_t [#338-mi_ai_bfinitattr_t]	定义音频波束成形功能的初始化信息结构体
MI_AI_BfConfigAttr_t [#339-mi_ai_bfconfigattr_t]	定义音频波束成形功能的配置信息结构体
MI_AI_InitParam_t [#340-mi_ai_initparam_t]	定义音频设备初始化参数
MI_AI_BabyCryConfig_t [#341-mi_ai_babycryconfig_t]	定义婴儿哭声检测的配置参数
MI_AI_LsdConfig_t [#342-mi_ai_lsdconfig_t]	定义高分贝检测的配置参数

3.2. MI_AUDIO_DEV

- 说明
定义音频输入/输出设备编号。
- 定义

```
typedef MI_S32 MI_AUDIO_DEV
```

3.3. MI_AUDIO_MAX_CHN_NUM

- 说明
定义音频输入/输出设备的最大通道数。
- 定义

```
#define MI_AUDIO_MAX_CHN_NUM 16
```

3.4. MI_AI_CHN

- 说明
定义音频输入通道。
- 定义

```
typedef MI_S32 MI_AI_CHN
```

3.5. MI_AUDIO_SampleRate_e

- 说明
定义音频采样率。
- 定义

```
typedef enum
{
    E_MI_AUDIO_SAMPLE_RATE_8000 = 8000, /* 8kHz sampling rate
    */
    E_MI_AUDIO_SAMPLE_RATE_11052 = 11025, /* 11.025kHz sampling
    rate */
    E_MI_AUDIO_SAMPLE_RATE_12000 = 12000, /* 12kHz sampling
    rate */
    E_MI_AUDIO_SAMPLE_RATE_16000 = 16000, /* 16kHz sampling
    rate */
    E_MI_AUDIO_SAMPLE_RATE_22050 = 22050, /* 22.05kHz sampling
    rate */
    E_MI_AUDIO_SAMPLE_RATE_24000 = 24000, /* 24kHz sampling
    rate */
}
```

```
E_MI_AUDIO_SAMPLE_RATE_32000 = 32000, /* 32kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_44100 = 44100, /* 44.1kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_48000 = 48000, /* 48kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_96000 = 96000, /* 96kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_INVALID,

}MI_AUDIO_SampleRate_e;
```

- 成员

表3-4

成员名称	描述
E_MI_AUDIO_SAMPLE_RATE_8000	8kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_11025	11.025kHz采样率
E_MI_AUDIO_SAMPLE_RATE_12000	12kHz采样率
E_MI_AUDIO_SAMPLE_RATE_16000	16kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_22050	22.05kHz采样率
E_MI_AUDIO_SAMPLE_RATE_24000	24kHz采样率
E_MI_AUDIO_SAMPLE_RATE_32000	32kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_44100	44.1kHz采样率
E_MI_AUDIO_SAMPLE_RATE_48000	48kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_96000	96kHz采样率

- 注意事项
 - 这里枚举值不是从0开始，而是与实际的采样率值相同。
 - AI仅支持8/16/32/48kHz。
- 相关数据类型及接口

MI_AUDIO_Attr_t [#314-mi_audio_attr_t]。

3.6. MI_AUDIO_Bitwidth_e

- 说明
定义音频采样精度。
- 定义

```
typedef enum
{
    E_MI_AUDIO_BIT_WIDTH_16 =0, /* 16bit width */
    E_MI_AUDIO_BIT_WIDTH_24 =1, /* 24bit width */
    E_MI_AUDIO_BIT_WIDTH_32 =2, /* 32bit width */
    E_MI_AUDIO_BIT_WIDTH_MAX,
}MI_AUDIO_BitWidth_e;
```

- 成员

表3-5

成员名称	描述
E_MI_AUDIO_BIT_WIDTH_16	采样精度为16bit 位宽
E_MI_AUDIO_BIT_WIDTH_24	采样精度为24bit 位宽
E_MI_AUDIO_BIT_WIDTH_32	采样精度为32bit 位宽

- 注意事项
目前芯片只支持16bit 位宽，但部分芯片的I2S可以支持32bit收发。

3.7. MI_AUDIO_Mode_e

- 说明
定义音频输入输出设备工作模式。
- 定义

```
typedef enum
{
    E_MI_AUDIO_MODE_I2S_MASTER, /* I2S master mode */
    E_MI_AUDIO_MODE_I2S_SLAVE, /* I2S slave mode */
    E_MI_AUDIO_MODE_TDM_MASTER, /* TDM master mode */
    E_MI_AUDIO_MODE_TDM_SLAVE, /* TDM slave mode */
    E_MI_AUDIO_MODE_MAX,
}MI_AUDIO_Mode_e;
```

- 成员

表3-6

成员名称	描述
E_MI_AUDIO_MODE_I2S_MASTER	I2S主模式
E_MI_AUDIO_MODE_I2S_SLAVE	I2S从模式
E_MI_AUDIO_MODE_TDM_MASTER	TDM主模式
E_MI_AUDIO_MODE_TDM_SLAVE	TDM从模式

- 注意事项
主模式与从模式是否支持会依据不同的芯片而有区别。
- 相关数据类型及接口

[MI_AUDIO_Attr_t](#) [#314-mi_audio_attr_t]

3.8. MI_AUDIO_SoundMode_e

- 说明
定义音频声道模式。
- 定义

```
typedef enum
{
    E_MI_AUDIO_SOUND_MODE_MONO =0, /* mono */
    E_MI_AUDIO_SOUND_MODE_STEREO =1, /* stereo */
    E_MI_AUDIO_SOUND_MODE_QUEUE =2, /*all data in One chn */
    E_MI_AUDIO_SOUND_MODE_MAX,
}MI_AUDIO_SoundMode_e
```

- 成员

表3-7

成员名称	描述
E_MI_AUDIO_SOUND_MODE_MONO	单声道
E_MI_AUDIO_SOUND_MODE_STEREO	双声道
E_MI_AUDIO_SOUND_MODE_QUEUE	所有音频数据按通道顺序排列在1个通道的缓冲区（针对音频采集时使用）

- 注意事项

当sound mode为E_MI_AUDIO_SOUND_MODE_QUEUE时，只需要对通道0做参数设置，其他通道会使用与通道0相同的参数，但是各个通道的处理流程相互独立。同时各个通道增益也是相互独立，需要用户分别设置。假设目前是4个通道，用户拿到的buffer大小为4nByte，则各个通道的数据量分别为4n / 4 = nByte。数据排列如下：

Ch0，Ch0，Ch0，Ch0，Ch0，Ch0，Ch0，Ch0.....

Ch1，Ch1，Ch1，Ch1，Ch1，Ch1，Ch1，Ch1.....

Ch2，Ch2，Ch2，Ch2，Ch2，Ch2，Ch2，Ch2.....

Ch3，Ch3，Ch3，Ch3，Ch3，Ch3，Ch3，Ch3.....

- 相关数据类型及接口

[MI_AUDIO_Attr_t](#) [#314-mi_audio_attr_t]

3.9. MI_AUDIO_AencType_e

- 说明

定义音频编码类型。

- 定义

```
typedef enum
{
    E_MI_AUDIO_AENC_TYPE_G711A = 0,
    E_MI_AUDIO_AENC_TYPE_G711U,
    E_MI_AUDIO_AENC_TYPE_G726,
    E_MI_AUDIO_AENC_TYPE_INVALID,
}MI_AUDIO_AencType_e;
```

- 成员

表3-8

成员名称	描述
E_MI_AUDIO_AENC_TYPE_G711A	G711A 编码。
E_MI_AUDIO_AENC_TYPE_G711U	G711U 编码。
E_MI_AUDIO_AENC_TYPE_G726	G726 编码。

- 相关数据类型及接口

[MI_AUDIO_AencG726Config_t](#) [#330-mi_audio_aencg726config_t]

3.10. MI_AUDIO_G726Mode_e

- 说明

定义G726工作模式。

- 定义

```
typedef enum
{
    E_MI_AUDIO_G726_MODE_16 = 0,
    E_MI_AUDIO_G726_MODE_24,
    E_MI_AUDIO_G726_MODE_32,
    E_MI_AUDIO_G726_MODE_40,
    E_MI_AUDIO_G726_MODE_INVALID,
}MI_AUDIO_G726Mode_e;
```

- 成员

表3-9

成员名称	描述
E_MI_AUDIO_G726_MODE_16	G726 16K 比特率模式。
E_MI_AUDIO_G726_MODE_24	G726 24K 比特率模式。
E_MI_AUDIO_G726_MODE_32	G726 32K 比特率模式。
E_MI_AUDIO_G726_MODE_40	G726 40K 比特率模式。

- 相关数据类型及接口

[MI_AUDIO_AencG726Config_t](#) [#330-mi_audio_aencg726config_t]

3.11. MI_AUDIO_I2sFmt_e

- 说明

I2S 格式设定。

- 定义

```
typedef enum
{
    E_MI_AUDIO_I2S_FMT_I2S_MSB,
    E_MI_AUDIO_I2S_FMT_LEFT_JUSTIFY_MSB,
}MI_AUDIO_I2sFmt_e;
```

- 成员

表3-10

成员名称	描述
E_MI_AUDIO_I2S_FMT_I2S_MSB	I2S 标准格式，最高位优先
E_MI_AUDIO_I2S_FMT_LEFT_JUSTIFY_MSB	I2S 左对齐格式，最高位优先

- 相关数据类型及接口

[MI_AUDIO_I2sConfig_t](#) [#313-mi_audio_i2sconfig_t]

3.12. MI_AUDIO_I2sMclk_e

- 说明

I2S MCLK 设定

- 定义

```
typedef enum{  
  
    E_MI_AUDIO_I2S_MCLK_0,  
  
    E_MI_AUDIO_I2S_MCLK_12_288M,  
  
    E_MI_AUDIO_I2S_MCLK_16_384M,  
  
    E_MI_AUDIO_I2S_MCLK_18_432M,  
  
    E_MI_AUDIO_I2S_MCLK_24_576M,  
  
    E_MI_AUDIO_I2S_MCLK_24M,  
  
    E_MI_AUDIO_I2S_MCLK_48M,  
  
}MI_AUDIO_I2sMclk_e;
```

- 成员

表3-11

成员名称	描述
E_MI_AUDIO_I2S_MCLK_0	关闭MCLK
E_MI_AUDIO_I2S_MCLK_12_288M	设置MCLK为12.88M
E_MI_AUDIO_I2S_MCLK_16_384M	设置MCLK为16.384M
E_MI_AUDIO_I2S_MCLK_18_432M	设置MCLK为18.432M
E_MI_AUDIO_I2S_MCLK_24_576M	设置MCLK为24.576M
E_MI_AUDIO_I2S_MCLK_24M	设置MCLK为24M
E_MI_AUDIO_I2S_MCLK_48M	设置MCLK为48M

- 相关数据类型及接口

[MI_AUDIO_I2sConfig_t](#) [#313-mi_audio_i2sconfig_t]

3.13. MI_AUDIO_I2sConfig_t

- 说明

定义I2S属性结构体。

- 定义

```
typedef struct MI_AUDIO_I2sConfig_s
{
    MI_AUDIO_I2sFmt_e eFmt;

    MI_AUDIO_I2sMclk_e eMclk;

    MI_BOOL bSyncClock;

    MI_U32 u32TdmSlots;

    MI_AUDIO_BitWidth_e eI2sBitWidth;
```

```
}MI_AUDIO_I2sConfig_t;
```

- 成员

表3-12

成员名称	描述
eFmt	I2S 格式设置 静态属性
eMclk	I2S MCLK 时钟设置 静态属性
bSyncClock	I2S Rx和I2s Tx是否共用clock。 静态属性
u32TdmSlots	I2S TDM slot的数目（仅TDM模式下有效）
eI2sBitWidth	I2S 收发的数据位宽（仅支持TDM模式的芯片有效）

- 相关数据类型及接口

[MI_AUDIO_Attr_t](#) [#314-mi_audio_attr_t]

3.14. MI_AUDIO_Attr_t

- 说明

定义音频输入输出设备属性结构体。

- 定义

```
typedef struct MI_AUDIO_Attr_s
{
    MI_AUDIO_SampleRate_e eSamplerate; /*sample rate*/
    MI_AUDIO_BitWidth_e eBitwidth; /*bitwidth*/
}
```

```

MI_AUDIO_Mode_e eWorkmode; /*master or slave mode*/

MI_AUDIO_SoundMode_e eSoundmode; /*momo or stereo*/

MI_U32 u32FrmNum; /*frame num in buffer*/

MI_U32 u32PtNumPerFrm; /*number of samples*/

MI_U32 u32CodecChnCnt; /*channel number on Codec */

MI_U32 u32ChnCnt;

    union{

        MI_AUDIO_I2sConfig_t stI2sConfig;

    }WorkModeSetting;

}MI_AUDIO_Attr_t;

```

- 成员

表3-13

成员名称	描述
eSamplerate	音频采样率 静态属性
eBitwidth	音频采样精度（从模式下，此参数必须和音频AD/DA的采样精度匹配） 静态属性
eWorkmode	音频I2S的工作模式 静态属性
eSoundmode	音频声道的数据格式 静态属性
u32FrmNum	缓存帧数目 保留，未使用
u32PtNumPerFrm	每帧的采样点个数。 取值范围为：128, 128*2,...,128*N 静态属性 注意：如不使用sigmastar提供的音频算法，可不必128对齐。
u32CodecChnCnt	支持的codec通道数目 保留，未使用
u32ChnCnt	支持的通道数目，实际可使能的最大通道数。取值：1、2、4、8、16。（输入最多支持MI_AUDIO_MAX_CHN_NUM个通道，输出最多支持2个通道）
MI_AUDIO_I2sConfig_t stI2sConfig;	设置I2S工作属性

- 相关数据类型及接口

[MI_AI_SetPubAttr](#) [#22-mi_ai_setpubattr]

3.15. MI_AI_ChnParam_t

- 说明

定义通道参数结构体。

- 定义

```
typedef struct MI_AI_ChnParam_s
{
    MI_AI_ChnGainConfig_t stChnGain;

    MI_U32 u32Reserved;
} MI_AI_ChnParam_t
```

- 成员

表3-14

成员名称	描述
stChnGain	音频通道的增益设定
u32Reserved	保留，未使用

- 相关数据类型及接口

[MI_AI_SetChnParam](#) [#210-mi_ai_setchnparam]

[MI_AI_GetChnParam](#) [#211-mi_ai_getchnparam]

3.16. MI_AUDIO_Frame_t

- 说明

定义音频帧结构体。

- 定义

```
MI_A0 API Version <= 2.9:

typedef struct MI_AUDIO_Frame_s
```

```

{

    MI_AUDIO_BitWidth_e eBitwidth;

    MI_AUDIO_SoundMode_e eSoundmode;

    void *apVirAddr[MI_AUDIO_MAX_CHN_NUM];

    MI_U64 u64TimeStamp;

    MI_U32 u32Seq;

    MI_U32 u32Len;

    MI_U32 au32PoolId[2];

}MI_AUDIO_Frame_t;

```

MI_A0 API Version 为 2.9 ~ 2.14:

```
typedef struct MI_AUDIO_Frame_s
```

```

{

    MI_AUDIO_BitWidth_e eBitwidth;

    MI_AUDIO_SoundMode_e eSoundmode;

    void *apVirAddr[MI_AUDIO_MAX_CHN_NUM];

    MI_U64 u64TimeStamp;

    MI_U32 u32Seq;

    MI_U32 u32Len;

    MI_U32 au32PoolId[2];

    void *apSrcPcmVirAddr[MI_AUDIO_MAX_CHN_NUM];

    MI_U32 u32SrcPcmLen;

}MI_AUDIO_Frame_t;

```

MI_A0 API Version >= 2.15:

```
typedef struct MI_AUDIO_Frame_s
```



```

{

    MI_AUDIO_BitWidth_e eBitwidth;

    MI_AUDIO_SoundMode_e eSoundmode;

    void *apVirAddr[MI_AUDIO_MAX_CHN_NUM];

    MI_U64 u64TimeStamp;

    MI_U32 u32Seq;

    MI_U32 u32Len[MI_AUDIO_MAX_CHN_NUM];

    MI_U32 au32PoolId[2];

    void *apSrcPcmVirAddr[MI_AUDIO_MAX_CHN_NUM];

    MI_U32 u32SrcPcmLen[MI_AUDIO_MAX_CHN_NUM];

}MI_AUDIO_Frame_t;

```

- 成员

表3-15

成员名称	描述
eBitwidth	音频采样精度
eSoundmode	音频声道模式
apVirAddr	音频帧数据虚拟地址
u64TimeStamp	音频帧时间戳 以 μs 为单位
u32Seq	音频帧序号
u32Len	音频帧长度 以byte 为单位
u32PoolId	音频帧缓存池ID
apSrcPcmVirAddr	音频帧原始数据虚拟地址
u32SrcPcmLen	音频帧原始数据长度 以byte 为单位

- 注意事项

- 当MI_AO API Version ≤ 2.16 ，u32Len（音频帧长度）指整个缓冲区的数据长度。当MI_AO API Version ≥ 2.15 ，每个channel的缓冲数据量为u32Len[channel index]。
- 单声道数据直接存放，采样点数为u32PtNumPerFrm，长度为u32Len；立体声数据按左右声道交错存放，数据格式为L,R,L,R,L,R....., 每个声道采样点数为u32PtNumPerFrm，总长度为u32Len。当sound mode为queue mode时，数据按通道顺序存放，各个通道的数据长度为u32Len[channel index]。
- 当没有开启任何算法时，pVirAddr等价于apSrcPcmVirAddr，u32Len等价于u32SrcPcmLen。当有开启算法时，pVirAddr返回的是算法处理后的数据，apSrcPcmVirAddr返回的是Ai采集到的原始pcm数据。

3.17. MI_AUDIO_AecFrame_t

- 说明
定义音频回声抵消参考帧信息结构体。
- 定义

```
typedef struct MI_AUDIO_AecFrame_s
{
    MI_AUDIO_Frame_t stRefFrame; /* aec reference audio frame
    */
    MI_BOOL bValid; /* whether frame is valid */
}MI_AUDIO_AecFrame_t;
```

- 成员

表3-16

成员名称	描述
stRefFrame	回声抵消参考帧结构体
bValid	参考帧有效的标志。 取值范围： TRUE：参考帧有效。 FALSE：参考帧无效，无效时不能使用此参考帧进行回声抵消。

3.18. MI_AUDIO_SaveFileInfo_t

- 说明
定义音频保存文件功能配置信息结构体。
- 定义

```
typedef struct MI_AUDIO_SaveFileInfo_s
```

```
{

    MI_BOOL  bCfg;

    MI_U8  szFilePath[256];

    MI_U32  u32FileSize; /*in KB*/

} MI_AUDIO_SaveFileInfo_t
```

- 成员

表3-17

成员名称	描述
bCfg	配置使能开关
szFilePath	音频文件的保存路径
u32FileSize	文件大小，取值范围[1,10240]KB

- 相关数据类型及接口

[MI_AI_SaveFile](#) [#219-mi_ai_savefile]

3.19. MI_AI_VqeConfig_t

- 说明

定义音频输入声音质量增强配置信息结构体。

- 定义

```
typedef struct MI_AI_VqeConfig_s

{

    MI_BOOL  bHpfOpen;

    MI_BOOL  bAecOpen;
```

```
MI_BOOL bAnrOpen;

MI_BOOL bAgcOpen;

MI_BOOL bEqOpen;

MI_U32 u32ChnNum;

MI_S32 s32WorkSampleRate;

MI_S32 s32FrameSample;

MI_AUDIO_HpfConfig_t stHpfCfg;

MI_AI_AecConfig_t stAecCfg;

MI_AUDIO_AnrcConfig_t stAnrCfg;

MI_AUDIO_AgcConfig_t stAgcCfg;

MI_AUDIO_EqConfig_t stEqCfg;

}MI_AI_VqeConfig_t;
```

- 成员

表3-18

成员名称	描述
bHpfOpen	高通滤波功能是否使能标志
bAecOpen	回声抵消功能是否使能标志
bAnrOpen	语音降噪功能是否使能标志
bAgcOpen	自动增益控制功能是否使能标志
bEqOpen	均衡器功能是否使能标志
u32ChnNum	数据的声道数
s32WorkSampleRate	工作采样频率。该参数为内部功能算法工作采样率。取值范围：8kHz/16kHz。默认值为8kHz。
s32FrameSample	VQE的帧长，即采样点数目。只能设置128
stHpfCfg	高通滤波功能相关配置信息
stAecCfg	回声抵消功能相关配置信息
stAnrCfg	语音降噪功能相关配置信息
stAgcCfg	自动增益控制相关配置信息
stEqCfg	均衡器相关配置信息

3.20. MI_AUDIO_HpfConfig_t

- 说明
定义音频高通滤波功能配置信息结构体。
- 定义

```
typedef struct MI_AUDIO_HpfConfig_s
```

```
{

    MI_AUDIO_AlgorithmMode_e eMode;

    MI_AUDIO_HpfFreq_e eHpfFreq; /*freq to be
    processed*/

} MI_AUDIO_HpfConfig_t;
```

- 成员

表3-19

成员名称	描述
eMode	音频算法的运行模式
eHpfFreq	高通滤波截止频率选择。 80：截止频率为80Hz； 120：截止频率为120Hz； 150：截止频率为150Hz。 默认值 150。

- 相关数据类型及接口

[MI_AI_VqeConfig_t](#) [#319-mi_ai_vqeconfig_t]

3.21. MI_AUDIO_HpfFreq_e

- 说明

定义音频高通滤波截止频率。

- 定义

```
typedef enum

{

    E_MI_AUDIO_HPF_FREQ_80 = 80, /* 80Hz */

    E_MI_AUDIO_HPF_FREQ_120 = 120, /* 120Hz */

}
```

```
E_MI_AUDIO_HPF_FREQ_150 = 150, /* 150Hz */  
  
E_MI_AUDIO_HPF_FREQ_BUTT,  
  
} MI_AUDIO_HpfFreq_e;
```

- 成员

表3-20

成员名称	描述
E_MI_AUDIO_HPF_FREQ_80	截止频率为80Hz
E_MI_AUDIO_HPF_FREQ_120	截止频率为120Hz
E_MI_AUDIO_HPF_FREQ_150	截止频率为150Hz

- 注意事项

默认配置为150Hz

- 相关数据类型及接口

[MI_AI_VqeConfig_t](#) [#319-mi_ai_vqeconfig_t]

3.22. MI_AI_AecConfig_t

- 说明

定义音频回声抵消配置信息结构体。

- 定义

```
MI_AI_API Version <= 2.17:  
  
typedef struct MI_AI_AecConfig_s  
  
{  
  
    MI_BOOL bComfortNoiseEnable;
```



```

    MI_S16 s16DelaySample;

    MI_U32 u32AecSupfreq[6];

    MI_U32 u32AecSupIntensity[7];

    MI_S32 s32Reserved;

} MI_AI_AecConfig_t;

MI_AI API Version >= 2.18:

typedef struct MI_AI_AecConfig_s
{
    MI_BOOL bComfortNoiseEnable;

    MI_S16 s16DelaySample;

    MI_U32 u32AecSupfreq[6];

    MI_U32 u32AecSupIntensity[7];

    MI_U32 u32AecRatioThreshold;

    MI_U32 u32AecDistortionTestFreq[2];

    MI_S32 s32Reserved;

} MI_AI_AecConfig_t;

```

- 成员

表3-21

MI_AI API Version <= 2.17:

成员名称	描述
bComfortNoiseEnable	是否添加噪音。 0：不添加； 1：添加。
s16DelaySample	采样点样本延迟个数。 仅在AEC为立体声处理时有效，默认值为0。
u32AecSupfreq	回声消除保护频率范围，后1个数据必须大于等于前1个数据。 如：u32AecSupfreq[0] = 10, 则：u32AecSupfreq[1]必须大于等于10。 当前采样率对应的最高频率平均分成128份，频率范围则是对应多少份组成一个频带。 如：当前采样率为16K，对应的最大频率为8K，每一份为$8000 / 128 \approx 62.5\text{Hz}$，在推荐值{4,6,36, 49,50,51}的设定下，保护范围为{$0 \sim 4 * 62.5\text{Hz}$, $4 \sim 6 * 62.5\text{Hz}$, $6 \sim 36 * 62.5\text{Hz}$, $36 \sim 49 * 62.5\text{Hz}$, $49 \sim 50 * 62.5\text{Hz}$, $50 \sim 51 * 62.5\text{Hz}$, $51 \sim 127 * 62.5\text{Hz}$} = {0~250Hz, 250~375Hz, 375~2250Hz, 2250~3062.5Hz, 3062.5~3125Hz, 3125~3187.5Hz, 3187.5Hz~8000Hz} 范围[1,127]；步长1 推荐值{4,6,36,49,50,51}
u32AecSupIntensity	回音消除保护力度，数值越小保护效果越强。此参数与u32AecSupfreq相对应，u32AecSupIntensity[0]对应0~u32AecSupfreq[0], u32AecSupIntensity[1]对应u32AecSupfreq[0]~ u32AecSupfreq[1]，以此类推。 范围[0,15]；步长1 推荐值{5,4,4,5,10,10,10}

MI_AI API Version >= 2.18:

成员名称	描述
bComfortNoiseEnable	是否添加噪音。 0：不添加； 1：添加。
s16DelaySample	采样点样本延迟个数。仅在AEC为立体声处理时有效，默认值为0。
u32AecSupfreq	回声消除保护频率范围，后1个数据必须大于等于前1个数据。 如：u32AecSupfreq[0] = 10, 则：u32AecSupfreq[1]必须大于等于10。 当前采样率对应的最高频率平均分成128份，频率范围则是对应多少份组成一个频带。 如：当前采样率为16K，对应的最大频率为8K，每一份为$8000 / 128 \approx 62.5\text{Hz}$，在推荐值{4,6,36, 49,50,51}的设定下，保护范围为{0~4 * 62.5Hz, 4~6 * 62.5Hz, 6~36 * 62.5Hz, 36~49 * 62.5Hz, 49~50 * 62.5Hz, 50~51 * 62.5Hz, 51~127 * 62.5Hz} = {0~250Hz, 250~375Hz, 375~2250Hz, 2250~3062.5Hz, 3062.5~3125Hz, 3125~3187.5Hz, 3187.5Hz~8000Hz} 范围 [1,127]； 步长1 推荐值{4,6,36,49,50,51}
u32AecSupIntensity	回音消除保护力度，数值越小保护效果越强。此参数与u32AecSupfreq相对应，u32AecSupIntensity[0]对应0~u32AecSupfreq[0], u32AecSupIntensity[1]对应u32AecSupfreq[0]~ u32AecSupfreq[1]，以此类推。范围 [0,15]； 步长1 推荐值{5,4,4,5,10,10,10}
u32AecRatioThreshold	执行回声消除的阈值条件。回声消除算法默认情况下（即u32AecRatioThreshold为0时）会一直进行回声消除处理。某些场景下需要近端和远端的能量比达到多少倍时，直接不进行回声消除处理。 例：当希望近端比远端的能量大n dB后，不进行回声消除处理，则$u32AecRatioThreshold = 10^{(n/10)}$
u32AecDistortionTest	回声消除失真测试频点。某些情况下希望不对某个频点进行压抑。当u32AecDistortionTest为0时，执行正常回声消除处理。

- 相关数据类型及接口

3.23. MI_AUDIO_AnrcConfig_t

- 说明

定义音频语音降噪功能配置信息结构体。

- 定义

```
typedef struct MI_AUDIO_AnrcConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;

    MI_U32 u32NrIntensity;

    MI_U32 u32NrSmoothLevel;

    MI_AUDIO_NrSpeed_e eNrSpeed;
} MI_AUDIO_AnrcConfig_t;
```

- 成员

表3-22

成员名称	描述
eMode	音频算法的运行模式 注：Anr的模式选择将会在一定程度上影响Agc的功能
u32NrIntensity	降噪力度配置，配置值越大降噪力度越高，但同时也会带来细节音的丢失/损伤。范围[0,30]； 步长1默认值 20。
u32NrSmoothLevel	平滑化程度，值越大越平滑 范围[0,10]； 步长1 默认值10。
eNrSpeed	噪声收敛速度，低速，中速，高速 默认值中速。

- 注意事项

在Anr和Agc都有使能的情况下，当Anr设定为user mode时，Agc会对音频数据做频域处理，会评估出语音信号再做相应的增减，而当Anr设定为default/music mode时，Agc会对音频数据做时域处理，对全频段的数据进行增减。

- 相关数据类型及接口

[MI_AI_VqeConfig_t](#) [#319-mi_ai_vqeconfig_t]

3.24. MI_AUDIO_NrSpeed_e

- 说明

定义噪声收敛速度

- 定义

```
typedef enum
{
    E_MI_AUDIO_NR_SPEED_LOW,
    E_MI_AUDIO_NR_SPEED_MID,
```

```
E_MI_AUDIO_NR_SPEED_HIGH

}MI_AUDIO_NrSpeed_e;
```

- 成员

表3-23

成员名称	描述
E_MI_AUDIO_NR_SPEED_LOW	低速
E_MI_AUDIO_NR_SPEED_MID	中速
E_MI_AUDIO_NR_SPEED_HIGH	高速

- 相关数据类型及接口

[MI_AI_VqeConfig_t](#) [#319-mi_ai_vqeconfig_t]

3.25. MI_AUDIO_AgcConfig_t

- 说明

定义音频自动增益控制配置信息结构体。

- 定义

因APC算法库更新，为兼容新旧版本的算法库，以下为不同版本对应的数据结构（可通过strings libAPC_LINUX.so | grep APCSET来获取APC算法库的版本，其中APC API版本会记录在APIVERSION字段中）

APC API VERSION:1（或没有APIVERSION字段）

```
typedef struct MI_AUDIO_AgcConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;
    AgcGainInfo_t stAgcGainInfo;
    MI_U32          u32DropGainMax;
    MI_U32          u32AttackTime;
    MI_U32          u32ReleaseTime;
```

```

MI_S16      s16Compression_ratio_input[5];
MI_S16      s16Compression_ratio_output[5];
MI_S32      s32TargetLevelDb;
MI_S32      s32NoiseGateDb;
MI_U32      u32NoiseGateAttenuationDb;
} MI_AUDIO_AgcConfig_t;

```

APC API VERSION:2

```

typedef struct MI_AUDIO_AgcConfig_s
{
MI_AUDIO_AlgorithmMode_e eMode;
    AgcGainInfo_t stAgcGainInfo;
    MI_U32      u32DropGainMax;
    MI_U32      u32AttackTime;
    MI_U32      u32ReleaseTime;
MI_S16      s16Compression_ratio_input[7];
    MI_S16      s16Compression_ratio_output[7];
    MI_S32      s32DropGainThreshold;
    MI_S32      s32NoiseGateDb;
    MI_U32      u32NoiseGateAttenuationDb;
} MI_AUDIO_AgcConfig_t;

```

- 成员

表3-24

APC API VERSION:1 (或没有APIVERSION字段)

成员名称	描述
eMode	音频算法的运行模式
stAgcGainInfo	定义AGC增益的最大、最小和初始值
u32DropGainMax	增益下降的最大值，防止输出饱和。范围[0,60]；步长1 默认值55。注意：此值仅代表能下降的范围，但具体能下降到何值还需参考AGC增益的最小值。
u32AttackTime	增益下降的时间步长，以16毫秒为1单位 范围[1,20]；步长1 默认值0。
u32ReleaseTime	增益增加的时间步长，以16毫秒为1单位 范围[1, 20]；步长1 默认值 0。
s16Compression_ratio_input	输入压缩比，必须配合s16Compression_ratio_output使用，透过多个转折点实现多斜率的曲线 范围[-80,0]dBFS；步长1
s16Compression_ratio_output	输出压缩比，必须配合s16Compression_ratio_input使用，透过多个转折点实现多斜率的曲线 范围[-80,0]dBFS；步长1
s32TargetLevelDb	目标电平，经过处理后最大能达到的门限电平。范围[-80,0]dB；步长1 默认值0。注意：此值仅能决定最大能达到的电平，但是否能达到此值，还需参考斜率曲线的设置。
s32NoiseGateDb	噪声阈值，当信号小于此值时，当作噪声处理 范围[-80,0]；步长1 默认值-55。注意：当值为-80，噪声阈值将不起作用
u32NoiseGateAttenuationDb	当噪声阈值起效果时，输入源的衰减百分比 范围[0,100]；步长1 默认值0。

APC API VERSION:2

成员名称	描述
eMode	音频算法的运行模式
stAgcGainInfo	定义AGC增益的最大、最小和初始值
u32DropGainMax	增益下降的最大值，防止输出饱和。范围[0,60]；步长1 默认值55。注意：此值仅代表能下降的范围，但具体能下降到何值还需参考AGC增益的最小值。
u32AttackTime	增益下降时间步长，以16毫秒为1单位 范围[1,20]；步长1 默认值0。
u32ReleaseTime	增益增加时间步长，以16毫秒为1单位 范围[1, 20]；步长1 默认值 0。
s16Compression_ratio_input	输入压缩比，必须配合s16Compression_ratio_output使用，透过多个转折点实现多斜率的曲线 范围[-80,0]dBFS；步长1
s16Compression_ratio_output	输出压缩比，必须配合s16Compression_ratio_input使用，透过多个转折点实现多斜率的曲线 范围[-80,0]dBFS；步长1
s32DropGainThreshold	衰减阈值，当信号幅度超过此值后，会慢慢进行衰减。范围[-80,0]dB；步长1 默认值0。
s32NoiseGateDb	噪声阈值，当信号小于此值时，当作噪声处理 范围[-80,0]；步长1 默认值-55。注意：当值为-80，噪声阈值将不起作用
u32NoiseGateAttenuationDb	当噪声阈值起效果时，输入源的衰减百分比 范围[0,100]；步长1 默认值0。

- 注意事项

在Anr和Agc都有使能的情况下，当Anr设定为user mode时，Agc会对音频数据做频域处理，会评估出语音信号再做相应的增减，而当Anr设定为default/music mode时，Agc会对音频数据做时域处理，对全频段的数据进行增减。

而s16Compression_ratio_input和s16Compression_ratio_output则需要根据所需要的增益曲线来设定。

如下面的折线图所示，在输入增益为-80~0dB划分为四段斜率，-80dB~-60dB范围内保持原来的增益，斜率为1，-60dB~-40dB范围内需要稍微提高增益，斜率为1.5，-40dB~-20dB范围内斜率为1.25，-20dB~0dB范围内斜率为0.25。根据曲线的转折点对s16Compression_ratio_input和s16Compression_ratio_output设置，若不需要那么多段曲线，则将数组不需要的部分填0。

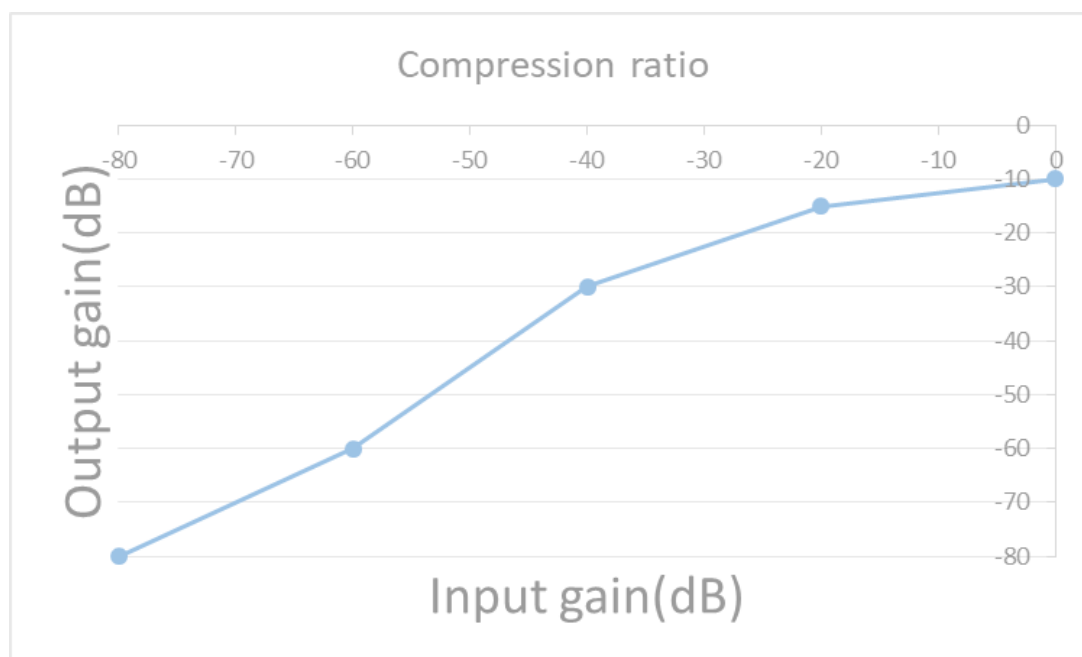


图3-1

- 相关数据类型及接口

[MI_AI_VqeConfig_t](#) [#319-mi_ai_vqeconfig_t]

3.26. AgcGainInfo_t

- 说明

AGC增益的取值

- 定义

```
typedef struct AgcGainInfo_s{
```

```
MI_S32 s32GainMax;

MI_S32 s32GainMin;

MI_S32 s32GainInit;

}AgcGainInfo_t;
```

- 成员

表3-25

成员名称	描述
s32GainMax	增益最大值 范围[0,60]；步长1 默认值15。
s32GainMin	增益最小值 范围[-20,30]；步长1 默认值0。
s32GainInit	增益初始值 范围[-20,60]；步长1 默认值0。

- 相关数据类型及接口

[MI_AI_VqeConfig_t](#) [#319-mi_ai_vqeconfig_t]

3.27. MI_AUDIO_EqConfig_t

- 说明

定义音频均衡器功能配置信息结构体。

- 定义

```
typedef struct MI_AUDIO_EqConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;

    MI_S16 s16EqGainDb[129];
} MI_AUDIO_EqConfig_t;
```

- 成员

表3-26

成员名称	描述
eMode	音频算法的运行模式
s16EqGainDb[129]	均衡器增益调节取值，将当前采样率的频率范围分成129个频率范围来进行调节 范围[-50,20]；步长1；默认值0。 如：当前采样率为16K，对应的最高频率为8K， $8000 / 129 \approx 62\text{Hz}$ ，则单个调节的频率范围为62Hz，将0-8K划分成 $\{0-1 * 62\text{Hz}, 1-2 * 62\text{Hz}, 2-3 * 62\text{Hz}, \dots, 128-129 * 62\text{Hz}\} = \{0-62\text{Hz}, 62-124\text{Hz}, 124-186\text{Hz}, \dots, 7938-8000\text{Hz}\}$ ，每段对应一个增益值

- 相关数据类型及接口

[MI_AI_VqeConfig_t](#) [#319-mi_ai_vqeconfig_t]

3.28. MI_AI_AencGConfig_t

- 说明

定义音频编码功能配置信息结构体。

- 定义

```
typedef struct MI_AI_AencConfig_s
{
    MI_AUDIO_AencType_e eAencType;

    union
    {
        MI_AUDIO_AencG711Config_t stAencG711Cfg;

        MI_AUDIO_AencG726Config_t stAencG726Cfg;
    };
};
```

```
}MI_AI_AencConfig_t;
```

- 成员

表3-27

成员名称	描述
eAencType	音频编码类型
stAencG711Cfg	G711编码相关配置信息
stAencG726Cfg	G726编码相关配置信息

- 相关数据类型及接口

[MI_AI_SetAencAttr](#) [#222-mi_ai_setaencattr]

3.29. MI_AUDIO_AencG711Config_t

- 说明

定义音频编码功能配置信息结构体。

- 定义

```
typedef struct MI_AUDIO_AencG711Config_s
{
    MI_AUDIO_SampleRate_e eSamplerate;
    MI_AUDIO_SoundMode_e eSoundmode;
}MI_AUDIO_AencG711Config_t;
```

- 成员

表3-28

成员名称	描述
eSamplerate	音频采样率。
eSoundmode	音频声道模式。

- 相关数据类型及接口

[MI_AI_SetAencAttr](#) [#222-mi_ai_setaencattr]

3.30. MI_AUDIO_AencG726Config_t

- 说明

定义音频编码功能配置信息结构体。

- 定义

```
typedef struct MI_AUDIO_AencG726Config_s
{
    MI_AUDIO_SampleRate_e eSamplerate;

    MI_AUDIO_SoundMode_e eSoundmode;

    MI_AUDIO_G726Mode_e eG726Mode;
}MI_AUDIO_AencG726Config_t;
```

- 成员

表3-29

成员名称	描述
eSamplerate	音频采样率
eSoundmode	音频声道模式
eG726Mode	G726工作模式

- 相关数据类型及接口

[MI_AI_SetAencAttr](#) [#222-mi_ai_setaencattr]

3.31. MI_AUDIO_AlgorithmMode_e

- 说明

音频算法运行的模式。

- 定义

```
typedef enum
{
    E_MI_AUDIO_ALGORITHM_MODE_DEFAULT,
    E_MI_AUDIO_ALGORITHM_MODE_USER,
    E_MI_AUDIO_ALGORITHM_MODE_MUSIC,
    E_MI_AUDIO_ALGORITHM_MODE_INVALID,
}MI_AUDIO_AlgorithmMode_e;
```

- 成员

表3-30

成员名称	描述
E_MI_AUDIO_ALGORITHM_MODE_DEFAULT	默认运行模式 当使用该模式时，将使用算法的默认参数
E_MI_AUDIO_ALGORITHM_MODE_USER	用户模式 当使用该模式时，需要用户重新设定所有参数
E_MI_AUDIO_ALGORITHM_MODE_MUSIC	音乐模式 仅有Anr具有此模式，当为此模式时，Agc不会进行speech enhancement（语音增强）处理

- 注意事项

在Anr和Agc都有使能的情况下，当Anr设定为user mode时，Agc会对音频数据做频域处理，会评估出语音信号再做相应的增减，而当Anr设定为default/music mode时，Agc会对音频数据做时域处理，对全频段的数据进行增减。

- 相关数据类型及接口

[MI_AUDIO_HpfConfig_t](#) [#320-mi_audio_hpfconfig_t]

[MI_AUDIO_AnrcConfig_t](#) [#323-mi_audio_anrconfig_t]

[MI_AUDIO_AgcConfig_t](#) [#325-mi_audio_agcconfig_t]

[MI_AUDIO_EqConfig_t](#) [#327-mi_audio_eqconfig_t]

3.32. MI_AI_AedConfig_t

- 说明

声音事件检测功能配置信息结构体。

- 定义

```
typedef struct MI_AI_AedConfig_s
{
    MI_BOOL bEnableNr;
```



```
MI_AUDIO_AedSensitivity_e eSensitivity;

MI_S32 s32OperatingPoint;

MI_S32 s32VadThresholdDb;

MI_S32 s32LsdThresholdDb;

}MI_AI_AedConfig_t;
```

- 成员

表3-31

成员名称	描述
bEnableNr	是否启用声音事件检测的降噪功能
eSensitivity	声音事件检测功能的灵敏度
s32OperatingPoint	操作点 范围[-10, 10]，步长为1 默认值为0 注：提高操作点将会降低误报率，减小操作点将会降低漏测率
s32VadThresholdDb	Vad的阈值(dB) 范围[-80, 0]，步长为1 默认值为-40
s32LsdThresholdDb	Lsd的阈值(dB) 范围[-80, 0]，步长为1 默认值为-15

- 相关数据类型及接口

[MI_AI_SetAedAttr](#) [#226-mi_ai_setaedattr]

[MI_AI_GetAedAttr](#) [#227-mi_ai_getaedattr]

3.33. MI_AUDIO_AedSensitivity_e/MI_AUDIO_BabyCrySensitivity_e

- 说明

婴儿哭声检测的灵敏度

- 定义

MI_AI API Version <= 2.15:

```
typedef enum
{
    E_MI_AUDIO_AED_SEN_LOW,
    E_MI_AUDIO_AED_SEN_MID,
    E_MI_AUDIO_AED_SEN_HIGH,
    E_MI_AUDIO_AED_SEN_INVALID,
}MI_AUDIO_AedSensitivity_e;
```

MI_AI API Version >= 2.16:

```
typedef enum
{
    E_MI_AUDIO_BABYCRY_SEN_LOW,
    E_MI_AUDIO_BABYCRY_SEN_MID,
    E_MI_AUDIO_BABYCRY_SEN_HIGH,
    E_MI_AUDIO_BABYCRY_SEN_INVALID,
} MI_AUDIO_BabyCrySensitivity_e;
```

- 成员

表3-32

成员名称	描述
E_MI_AUDIO_AED_SEN_LOW/ E_MI_AUDIO_BABYCRY_SEN_LOW	低灵敏度
E_MI_AUDIO_AED_SEN_MID/ E_MI_AUDIO_BABYCRY_SEN_MID	中等灵敏度
E_MI_AUDIO_AED_SEN_HIGH/ E_MI_AUDIO_BABYCRY_SEN_HIGH	高灵敏度

- 相关数据类型及接口

[MI_AI_AedConfig_t](#) [#332-mi_ai_aedconfig_t]

3.34. MI_AI_AedResult_t

- 说明

声音事件检测结果结构体。

- 定义

```
typedef struct MI_AI_AedResult_s
{
    MI_BOOL bAcousticEventDetected;

    MI_BOOL bLoudSoundDetected;

}MI_AI_AedResult_t;
```

- 成员

表3-33

成员名称	描述
bAcousticEventDetected	是否检测到声音事件
bLoudSoundDetected	是否检测到高分贝声音

- 相关数据类型及接口

[MI_AI_GetAedResult](#) [#230-mi_ai_getaedresult]

3.35. MI_AI_ChnGainConfig_t

- 说明

音频通道增益设置结构体。

- 定义

```
typedef struct MI_AI_ChnGainConfig_s
{
    MI_BOOL bEnableGainSet;

    MI_S16 s16FrontGain;

    MI_S16 s16RearGain;
```

```
}MI_AI_ChnGainConfig_t;
```

• 成员

表3-34

成员名称	描述
bEnableGainSet	是否使能增益设置
s16FrontGain	前级增益
s16RearGain	后级增益

• 注意事项

以下表格描述了各个device对s16FrontGain和s16RearGain的定义

表3-35

AI Device ID	s16FrontGain	s16RearGain
Amic	Amic的模拟增益(0 - 21)	Amic的数字增益(-60 - 30)
Dmic	Dmic的增益 Pretzel系列(0 - 4) Macaron系列(-60 - 30) Taiyaki系列(0 - 4) Takoyaki系列(0 - 4) Pudding系列(-60 - 30) Ispahan系列(-60 - 30) Ikayaki系列(0 - 4)	不起作用
I2S RX	不起作用	不起作用
Line in	Line in的模拟增益(0 - 7)	Line in的数字增益(-60 - 30)

• 相关数据类型及接口

[MI_AI_ChnParam_t](#) [#315-mi_ai_chnparam_t]

3.36. MI_AI_SslInitAttr_t

- 说明

音频声源定位功能初始化结构体。

- 定义

```
typedef struct MI_AI_SslInitAttr_s
{
    MI_U32 u32MicDistance;

    MI_BOOL bBfMode;
} MI_AI_SslInitAttr_t;
```

- 成员

表3-36

成员名称	描述
u32MicDistance	麦克风的间距（cm）步长为1。
bBfMode	是否为beamforming模式 FALSE为机械转动模式，TRUE为beamforming模式。

- 相关数据类型及接口

[MI_AI_SetSslInitAttr](#) [#232-mi_ai_setsslinitattr]

[MI_AI_GetSslInitAttr](#) [#233-mi_ai_getsslinitattr]

3.37. MI_AI_SslConfigAttr_t

- 说明

音频声源定位功能配置结构体。

- 定义

```
typedef struct MI_AI_SslConfigAttr_s
{
    MI_S32 s32Temperature;

    MI_S32 s32NoiseGateDbfs;

    MI_S32 s32DirectionFrameNum;

} MI_AI_SslConfigAttr_t;
```

- 成员

表3-37

成员名称	描述
s32Temperature	温度（摄氏度） 摄氏度 = (5/9)*(华氏度 - 32) 步长为1
s32NoiseGateDbfs	噪音增益门限值（dB） 注意：低于此值将会把该帧作为噪声部分处理。步长为1
s32DirectionFrameNum	声源定位功能检测的帧数 步长为50 注意：进行声源定位检测的帧数，数值必须为50的倍数，声源定位处理的一帧数据为128个采样点。 检测一次的时间 = s32DirectionFrameNum * 128 / 采样率。 如：当前采样率为48K，s32DirectionFrameNum设置成300，则检测时间=300 * 128 / 48000 = 0.8(s)

- 相关数据类型及接口

[MI_AI_SetSslConfigAttr](#) [#234-mi_ai_setsslconfigattr]
[MI_AI_GetSslConfigAttr](#) [#235-mi_ai_getsslconfigattr]

3.38. MI_AI_BfInitAttr_t

- 说明

音频波束成形功能初始化结构体。

- 定义

```
typedef struct MI_AI_BfInitAttr_s
{
    MI_U32 u32MicDistance;

    MI_U32 u32ChanCnt;

} MI_AI_BfInitAttr_t;
```

- 成员

表3-38

成员名称	描述
u32MicDistance	麦克风的间距（cm）步长为1。
u32ChanCnt	通道数。 注意：只能设置为2。

- 相关数据类型及接口

[MI_AI_SetBfInitAttr](#) [#239-mi_ai_setbfinitattr]

[MI_AI_GetBfInitAttr](#) [#240-mi_ai_getbfinitattr]

3.39. MI_AI_BfConfigAttr_t

- 说明

音频波束成形功能配置结构体。

- 定义

```
typedef struct MI_AI_BfConfigAttr_s
```

```
{  
  
    MI_S32 s32Temperature;  
  
    MI_S32 s32NoiseGateDbfs;  
  
    MI_S32 s32NoiseSupressionMode;  
  
    MI_S32 s32NoiseEstimation;  
  
    MI_FLOAT outputGain;  
  
} MI_AI_BfConfigAttr_t;
```

• 成员

表3-39

成员名称	描述
s32Temperature	温度（摄氏度） 摄氏度 = (5/9)*(华氏度 - 32) 步长为1
s32NoiseGateDbfs	噪音增益门限值（dB） 注意：低于此值将会把该帧作为噪声部分处理。如果此值设置得太高，将会导致语音失真，如果设置得太低，会影响BF的效果。推荐值为-44，步长为1
s32NoiseSupressionMode	噪声抑制模式 范围[0,15]，步长为1 推荐值为8 注意：此值越大，噪声消除强度越强。如果此值设置得太高，将会导致语音失真，如果设置得太低，会影响BF的效果。
s32NoiseEstimation	噪声的估计模式 取值为0或1。0为自适应模式 1为平均模式 关键词定位预处理推荐使用0 语音通讯推荐使用1
outputGain	输出增益 范围[0,1] 推荐值0.7 注意：设定为1，会增加4dB

• 相关数据类型及接口

[MI_AI_SetBfConfigAttr](#) [#241-mi_ai_setbfconfigattr]

[MI_AI_GetBfConfigAttr](#) [#242-mi_ai_getbfconfigattr]

3.40. MI_AI_InitParam_t

- 说明
AI设备初始化参数。
- 定义

```
typedef struct MI_AI_InitParam_s
{
    MI_U32 u32DevId;

    MI_U8 *u8Data;
} MI_AI_InitParam_t;
```

- 成员

表3-40

成员名称	描述
u32DevId	音频设备号
u8Data	参数指针（保留）

- 相关数据类型及接口
[MI_AI_InitDev](#) [#248-mi_ai_initdev]

3.41. MI_AI_BabyCryConfig_t

- 说明
婴儿哭声检测的配置参数。
- 定义

```
typedef struct MI_AI_BabyCryConfig_s
```

```
{

    MI_BOOL bEnableNr;

    MI_AUDIO_BabyCrySensitivity_e eSensitivity;

    MI_S32 s32OperatingPoint;

    MI_S32 s32VadThresholdDb;

} MI_AI_BabyCryConfig_t;
```

- 成员

表3-41

成员名称	描述
bEnableNr	是否启用婴儿哭声检测的降噪功能
eSensitivity	婴儿哭声检测的灵敏度
s32OperatingPoint	婴儿哭声检测的操作点 范围[-10, 10]，步长为1 默认值为0 注：提高操作点将会降低误报率，减小操作点将会降低漏测率
s32VadThresholdDb	婴儿哭声检测的Vad阈值(dB) 范围[-80, 0]，步长为1 默认值为-40

- 相关数据类型及接口

[MI_AI_AedConfig_t](#) [#332-mi_ai_aedconfig_t]

3.42. MI_AI_LsdConfig_t

- 说明

高分贝检测的配置参数。

- 定义

```
typedef struct MI_AI_LsdConfig_s
{
    MI_S32 s32LsdThresholdDb;
} MI_AI_LsdConfig_t;
```

- 成员

表3-42

成员名称	描述
s32LsdThresholdDb	Lsd的阈值(dB) 范围[-80, 0] · 步长为1 默认值为-15

- 相关数据类型及接口

[MI_AI_AedConfig_t](#) [#332-mi_ai_aedconfig_t]

4. 错误码

AI API 错误码如表4-1所示：

错误码	宏定义	描述
0xA0042001	MI_AI_ERR_INVALID_DEVID	音频输入设备号无效
0xA0042002	MI_AI_ERR_INVALID_CHNID	音频输入信道号无效
0xA0042003	MI_AI_ERR_ILLEGAL_PARAM	音频输入参数设置无效
0xA0042006	MI_AI_ERR_NULL_PTR	输入参数空指标错误
0xA0042007	MI_AI_ERR_NOT_CONFIG	音频输入设备属性未设置
0xA0042008	MI_AI_ERR_NOT_SUPPORT	操作不支持
0xA0042009	MI_AI_ERR_NOT_PERM	操作不允许
0xA004200C	MI_AI_ERR_NOMEM	分配内存失败或相应的数据队列已满
0xA004200D	MI_AI_ERR_NOBUF	音频输入缓存不足
0xA004200E	MI_AI_ERR_BUF_EMPTY	音频输入缓存为空
0xA004200F	MI_AI_ERR_BUF_FULL	音频输入缓存为满
0xA0042010	MI_AI_ERR_SYS_NOTREADY	音频输入系统未初始化
0xA0042012	MI_AI_ERR_BUSY	音频输入系统忙碌
0xA0042017	MI_AI_ERR_NOT_ENABLED	音频输入设备或通道没有使能
0xA0042100	MI_AI_ERR_VQE_ERR	音频输入VQE算法处理失败
0xA0042101	MI_AI_ERR_AENC_ERR	音频输入编码算法处理失败
0xA0042102	MI_AI_ERR_AED_ERR	声音检测算法处理失败
0xA0042103	MI_AI_ERR_SSL_ERR	声源定位算法处理失败
0xA0042104	MI_AI_ERR_BF_ERR	波束成形算法处理失败

5. PROCFS介绍

5.1. cat

- 调试信息

```
# cat proc/mi_modules/mi_ai/mi_ai0
```

```
===== MI AI Dev0 Info =====
----- Start AI Dev0 Attr -----
DevStatus   Format   SoundMode   SampleRate   PeriodSize   Interleaved
started     S16_LE   Mono        16000        1024         1

MhalChnCnt   MiChnGrpCnt   ReadCountFromMhal
2            2             170

If slot[0]: ADC_AB

ADC_AB:      Gain( 18, 18) Mute(0,0).
----- End AI Dev0 Attr -----
----- Start AI ChnGrp0 info -----
ChnGrpId    bEnable   ReadFrameCount
0           1         170
Mute Status:
Mute[0]: 0

Dpga Gain:
Dpga Gain[0]: 0

----- End AI ChnGrp0 info -----

----- Start AI ChnGrp1 info -----
ChnGrpId    bEnable   ReadFrameCount
1           1         170
Mute Status:
Mute[0]: 0

Dpga Gain:
Dpga Gain[0]: 0

----- End AI ChnGrp1 info -----
```

- 调试信息分析

记录当前AI的使用状况以及device属性、channel group属性，可以动态地获取到这些信息，方便调试和测试。

- 参数说明

参数		描述
AI Device Attr	DevStatus	AI设备状态 uninit:未初始化 opened:open成功
	Format	音频格式（位宽及大小端等） 目前仅支持S16_LE（16bit，小端模式）
	SoundMode	声音模式 mono: 单声道 stereo: 立体声 SOUND_MODE_4CH: 4channel SOUND_MODE_6CH: 6channel SOUND_MODE_8CH: 8channel
	SampleRate	采样率 8k/16k/32k/48k
	PeriodSize	AI每帧数据量
	Interleaved	是否为交错模式
	MhalChnCnt	mhal设备的通道数
	MiChnGrpCnt	MI channel group数
	ReadCountFromMhal	AI从mhal读取的帧数
	If slot[lflidx]: interface type	[lflidx]: Interface 索引 interface type: Interface类型
AI I2S Status (AI I2S信息)	[interface type]: Gain(Leftgain,RightGain) Mute(LeftMute,RightMute)	[interface type]: Interface类型 Gain(Leftgain,RightGain): 左声道增益，右声道增益 Mute(LeftMute,RightMute): 左声道静音，右声道静音
	I2sMode	I2S Rx的工作模式(仅interface为I2S Rx时有效) i2s-master

		i2s-master i2s-slave tdm-master tdm-slave
	I2sMclk	I2S Rx的Mclk的频率(仅interface为I2S Rx时有效) disable: 不使用Mclk 其他值为当前的Mclk频率
	I2sFmt	I2S Rx的数据格式(仅interface为I2S Rx时有效) I2S-MSB: I2S格式 LEFT-MSB: I2S左对齐格式
	bl2sSync	I2S RX和TX是否共用clock(仅interface为I2S Rx时有效) 1: 4 wire mode · RX和TX共用clock 0: 6 wire mode · RX和TX都有独立的clock
	TdmSlots	I2S Rx的TDM slot数目(仅interface为I2S Rx为TDM模式时有效)
	I2sBitWidth	I2S RX的位宽(仅interface为I2S Rx · 且需要支持TDM模式的芯片有效)
AI ChnGrp info (AI 通道信息)	ChnGrpId	MI channel group ID
	bEnable	是否使能
	ReadFrameCount	对应channel group读取的帧数
	Mute Status	DPGA Mute信息
	Mute[PhyChnIdx]:bMute	[PhyChnIdx]:channel group中的channel索引 bMute : 静音状态
	Dpga Gain	DPGA 增益信息
	Dpga Gain[PhyChnIdx]:Gain	[PhyChnIdx]: channel group中的channel索引

Gain:增益

5.2. echo

功能	动态启用/关闭AI设备dump mhal pcm数据
命令	echo dump_mhal_chn [Path] [ON/on/1, OFF/off/0] > proc/mi_modules/mi_ao/mi_ac
参数说明	[Path] dump文件保存的路径 [ON/on/1, OFF/off/0] 是否使能 [ID] AI设备号
举例	echo dump_mhal_chn /tmp 1 > proc/mi_modules/mi_ao/mi_ai0



功能	动态启用/关闭AI设备dump mi pcm数据
命令	echo dump_mi_chn [ChnGrpId] [Path] [ON/on/1, OFF/off/0] > proc/mi_modules/mi_
参数说明	[ChnGrpId] channel group ID [Path] dump文件保存的路径 [ON/on/1, OFF/off/0] 是否使能 [ID] AI设备号
举例	echo dump_mi_chn 0 /tmp 1 > proc/mi_modules/mi_ao/mi_ai0



功能	动态设定AI DPGA的增益
命令	<code>echo set_dpga_gain [ChnGrpId] [ChnId] [Gain] > proc/mi_modules/mi_ao/mi_ao[ID]</code>
参数说明	[ChnGrpId] channel group ID [ChnId] channel group中的channel索引 [Gain] AI数字增益
举例	<code>echo set_dpga_gain 0 0 -10 > proc/mi_modules/mi_ao/mi_ai0</code>



功能	动态设定AI Interface的增益
命令	<code>echo set_inf_gain [If] [LeftGain] [RightGain] > proc/mi_modules/mi_ao/mi_ao[ID]</code>
参数说明	[If] AI Interface [LeftGain] [RightGain] 左声道增益 · 右声道增益
举例	<code>echo set_inf_gain 1 18 18 > proc/mi_modules/mi_ao/mi_ai0</code>



功能	动态设定AI DPGA静音模式
命令	<code>echo set_dpga_mute [ChnGrpId] [ChnId] [Mute] > proc/mi_modules/mi_ao/mi_ao[ID]</code>
参数说明	[ChnGrpId] channel group ID [ChnId] channel group中的channel索引 [Mute] AI DPGA静音
举例	<code>echo set_dpga_mute 0 0 1 > proc/mi_modules/mi_ao/mi_ai0</code>



功能	动态设定AI Interface静音模式
命令	echo set_inf_mute [If] [LeftMute] [RightMute] > proc/mi_modules/mi_ao/mi_ao[ID]
Parameter	[If] AI Interface
Description	[LeftMute] [RightMute] Mute left channel, mute right channel
Example	echo set_inf_mute 1 0 1 > proc/mi_modules/mi_ao/mi_ai0

